

Retail Analytics: Predicting Customer Purchase Behavior

Problem Statement:

The goal of this capstone project is to develop a comprehensive understanding of customer behavior and preferences based on retail transaction data. The aim is to analyze customers' demographic information, purchasing patterns, and interactions with marketing campaigns to identify key insights. These insights will inform strategic decision-making and drive business growth. Specifically, the focus will be on predicting customers' responses to marketing campaigns and optimizing promotional strategies to maximize their effectiveness and return on investment (ROI).

Actions:

Excel Tasks:

1. Data Exploration:

Task 1: Create a statistical summary for numerical features

We will go to excel , we will convert table of marketing campaign , Go to sheet 1 then tap on Data -> Data Analysis -> Descriptive statistics

Select Descriptive analysis:

Select input Range :Income column

output option: New Worksheet

click on checkbox to yes for summary statistics

Rename head to Income

Same we have to create statistical analysis for all numerical features

(Recency, MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds, NumDealsPurchases, NumWebPurchases, NumCatalogPurchases,

NumStorePurchases, NumWebVisitsMonth, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5, AcceptedCmp1, AcceptedCmp2, Complain)

Project 1.2m - Export															
Standard Deviation															
Income		Recency		MntWines		MntFruits		NumDealPurchases		NumCatalogPurchases					
Mean	52247.25135	Mean	49.109175	Mean	305.9357143	Mean	26.5023214	Mean	2.525	Mean	2.662053571				
Standard Error	534.7507722	Standard Error	0.611943505	Standard Error	7.111917962	Standard Error	0.840367189	Standard Error	0.040825969	Standard Error	0.061761774				
Median	51381.5	Median	49	Median	173.5	Median	8	Median	2	Median	2				
Mode	2500	Mode	55	Mode	2	Mode	0	Mode	1	Mode	0				
Standard Deviation	25173.07669	Standard Deviation	28.96245281	Standard Deviation	336.3973928	Standard Deviation	39.77343276	Standard Deviation	1.932237501	Standard Deviation	2.923100056				
Sample Variance	633883788.6	Sample Variance	838.8236727	Sample Variance	113297.8047	Sample Variance	1581.926033	Sample Variance	3.73354176	Sample Variance	8.54517542				
Kurtosis	159.6366996	Kurtosis	-1.201806799	Kurtosis	0.598743594	Kurtosis	4.050976251	Kurtosis	8.036914321	Kurtosis	8.047436789				
Skewness	6.763487373	Skewness	-0.001986659	Skewness	1.175770564	Skewness	2.102063305	Skewness	2.418569388	Skewness	1.880988778				
Range	664936	Range	99	Range	1493	Range	199	Range	15	Range	28				
Minimum	1730	Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0				
Maximum	666666	Maximum	99	Maximum	1493	Maximum	199	Maximum	15	Maximum	28				
Sum	115779909	Sum	110005	Sum	680816	Sum	58917	Sum	5208	Sum	5963				
Count	2216	Count	2240	Count	2240	Count	2240	Count	2240	Count	2240				
1		1		1		1		1		1					
MntMeatProducts		MntFishProducts		MntSweetProducts		MntGoldProds		NumWebPurchases		NumStorePurchases					
Mean	166.95	Mean	37.52545643	Mean	27.06294643	Mean	41.021875	Mean	4.08482129	Mean	5.790178571				
Standard Error	4.769102686	Standard Error	1.154247859	Standard Error	0.872209723	Standard Error	1.102238324	Standard Error	0.058711052	Standard Error	0.068689028				
Median	0	Median	12	Median	0	Median	4	Median	4	Median	5				
Mode	7	Mode	0	Mode	0	Mode	1	Mode	2	Mode	3				
Standard Deviation	225.7153725	Standard Deviation	54.6289794	Standard Deviation	41.28049849	Standard Deviation	52.16743891	Standard Deviation	2.778714147	Standard Deviation	3.250958146				
Sample Variance	50947.42939	Sample Variance	2984.325391	Sample Variance	1704.079555	Sample Variance	2721.441683	Sample Variance	7.721252313	Sample Variance	10.56872886				
Kurtosis	5.516724101	Kurtosis	3.096460912	Kurtosis	4.376548261	Kurtosis	3.55170925	Kurtosis	5.703128364	Kurtosis	-0.622048277				
Skewness	2.083233131	Skewness	1.919768971	Skewness	2.136080712	Skewness	1.886105609	Skewness	1.382794296	Skewness	0.702237286				
Range	1725	Range	259	Range	263	Range	362	Range	27	Range	13				
Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0				
Maximum	1725	Maximum	259	Maximum	263	Maximum	362	Maximum	27	Maximum	13				
Sum	373968	Sum	84057	Sum	60621	Sum	98609	Sum	9150	Sum	12570				
Count	2240	Count	2240	Count	2240	Count	2240	Count	2240	Count	2240				
1		1		1		1		1		1					

Project 1.2m - Export															
Standard Deviation															
NumWebVisitMonths		AcceptedCmp4		AcceptedCmp1		Complain									
Mean	5.316517857	Mean	0.074553571	Mean	0.064285714	Mean	0.009375								
Standard Error	0.051272234	Standard Error	0.00551148	Standard Error	0.005183246	Standard Error	0.002036635								
Median	6	Median	0	Median	0	Median	0								
Mode	7	Mode	0	Mode	0	Mode	0								
Standard Deviation	2.42664501	Standard Deviation	0.262728285	Standard Deviation	0.245315974	Standard Deviation	0.096391188								
Sample Variance	5.888608002	Sample Variance	0.069026152	Sample Variance	0.060179927	Sample Variance	0.00921257								
Kurtosis	1.821613827	Kurtosis	8.515406332	Kurtosis	10.65069424	Kurtosis	101.9061347								
Skewness	0.207925557	Skewness	3.241574322	Skewness	3.55544485	Skewness	10.18897187								
Range	20	Range	1	Range	1	Range	1								
Minimum	0	Minimum	0	Minimum	0	Minimum	0								
Maximum	20	Maximum	1	Maximum	1	Maximum	1								
Sum	11909	Sum	167	Sum	144	Sum	21								
Count	2240	Count	2240	Count	2240	Count	2240								
1		1		1		1									
AcceptedCmp3		AcceptedCmp5		AcceptedCmp2											
Mean	0.072767857	Mean	0.072767857	Mean	0.013392857										
Standard Error	0.005489553	Standard Error	0.005489553	Standard Error	0.002429303										
Median	0	Median	0	Median	0										
Mode	0	Mode	0	Mode	0										
Standard Deviation	0.25981307	Standard Deviation	0.25981307	Standard Deviation	0.114975606										
Sample Variance	0.067502833	Sample Variance	0.067502833	Sample Variance	0.013219399										
Kurtosis	8.843214224	Kurtosis	8.843214224	Kurtosis	69.83871128										
Skewness	3.291704618	Skewness	3.291704618	Skewness	8.472092779										
Range	1	Range	1	Range	1										
Minimum	0	Minimum	0	Minimum	0										
Maximum	1	Maximum	1	Maximum	1										
Sum	163	Sum	163	Sum	30										
Count	2240	Count	2240	Count	2240										
1		1		1											

Task 2: Create a line chart for the number of enrolments by year

Insert pivot table

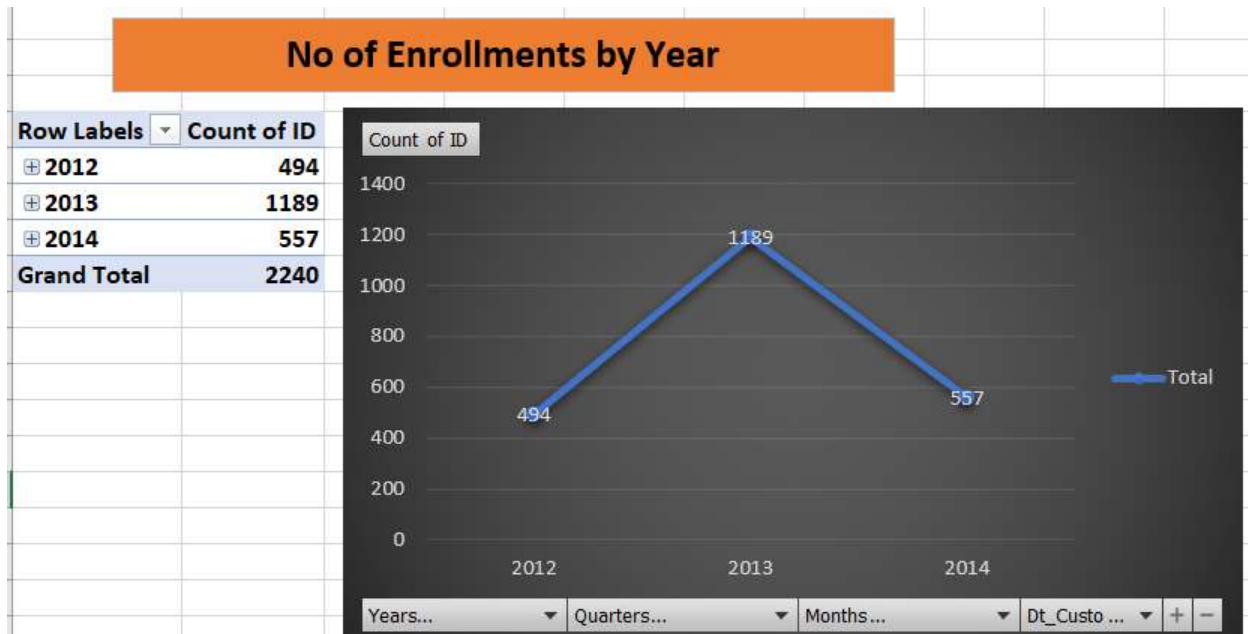
Drag Id to sum of values area and by default it will be sum change to count.

Drag Dt_Customer to Rows

Then Insert line chart

Format the chart

Insert text box for heading

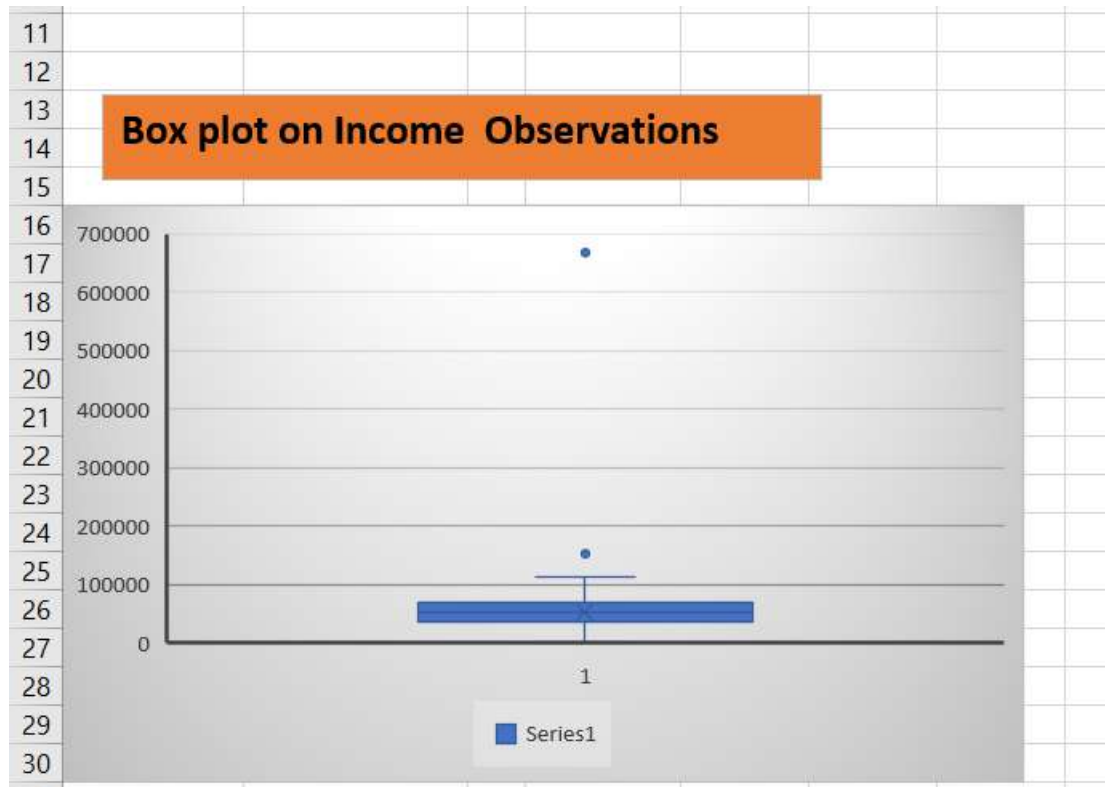


Task 3: Give a cross-tabulated count for response values against education

Count of Responses Against Education				
Count of ID	Column Labels			
Row Labels	No	Yes	Grand Total	
2n Cycle	181	22	203	
Basic	52	2	54	
Graduation	975	152	1127	
Master	313	57	370	
PhD	385	101	486	
Grand Total	1906	334	2240	

Task 4: Make a boxplot on income and write your observations

Select Income Column from data ,Insert Box plot chart then copy paste in another sheet and format it



Some People have extremely high salary , but many people have a average salary of 50000\$.75th percentile is around 68592 something,25th percentile is around 35625 something and middle value is \$51000 something.

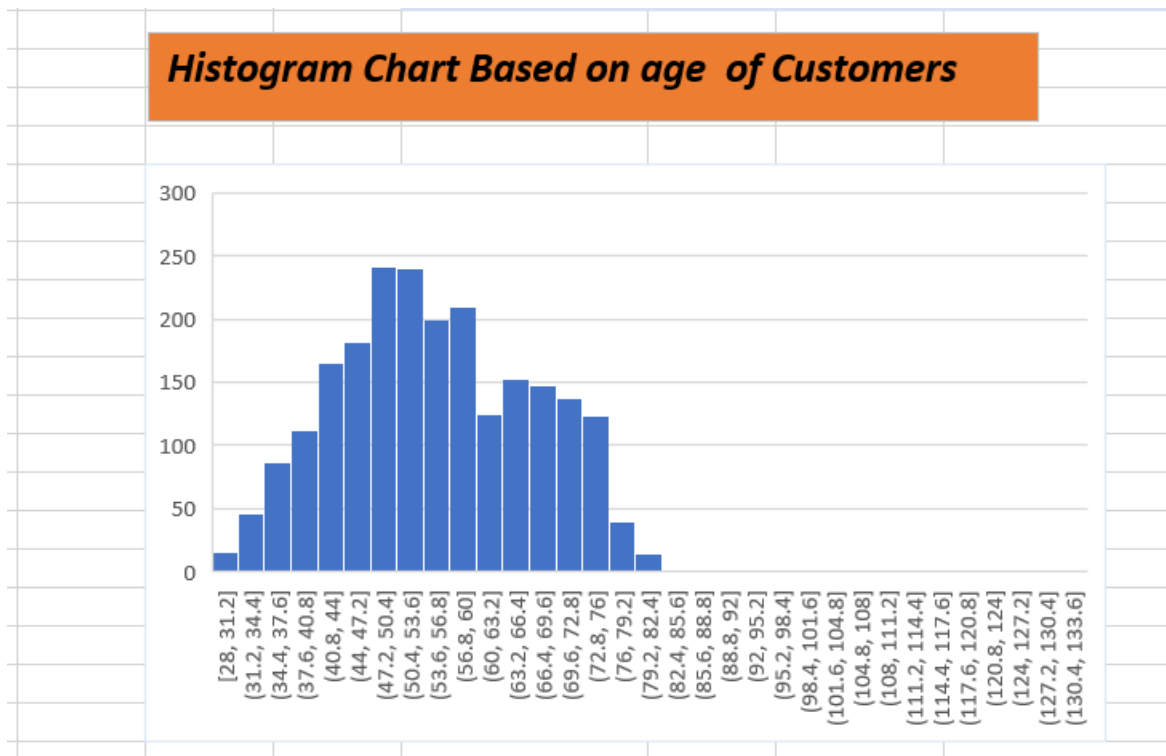
Task 5: Calculate the age of customers and make a histogram of that

Select age column from data

Insert histogram chart

Format it

Insert text box and name it as histogram chart based on age of customers



We can observe from chart that maximum customers are age group between 47 to 56 age group .there are very few people from age group of 28 to 31. There are no response from age group below 28 and above 82 .

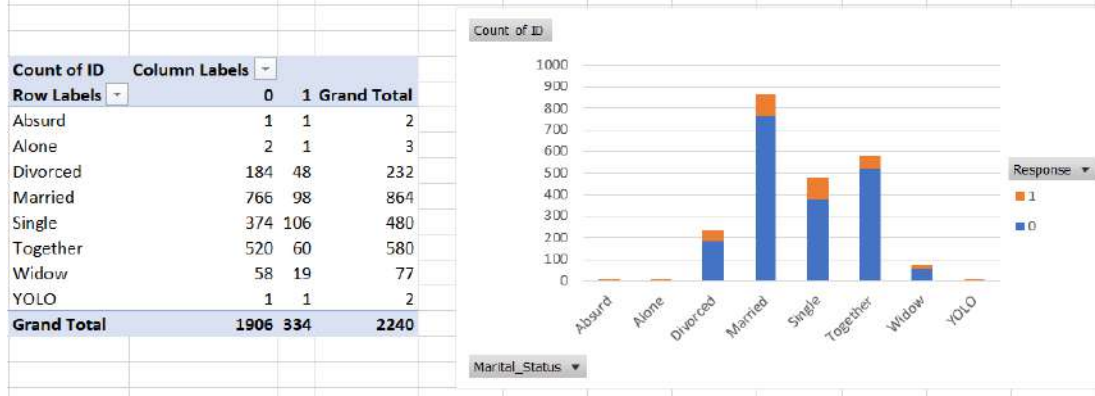
Task 6: Visualize the response against Marital_Status

Insert pivot table

Drag Marital status to rows, Drag response to columns and id to sum of values where you have to changed id to count then insert Stack column

We have more no of responses from married people -766 and lowest from widow ie.19 and there are very less response almost negligible from Absurd, alone and yolo

Responses Against Marital Status



Total respondents: 2,240 — of which **1,906** responded "0" and only **334** responded "1", meaning roughly **85% responded 0** vs **15% responded 1**.

Married individuals are the highest responders (864), nearly **1.5x more** than the next group (Together: 580).

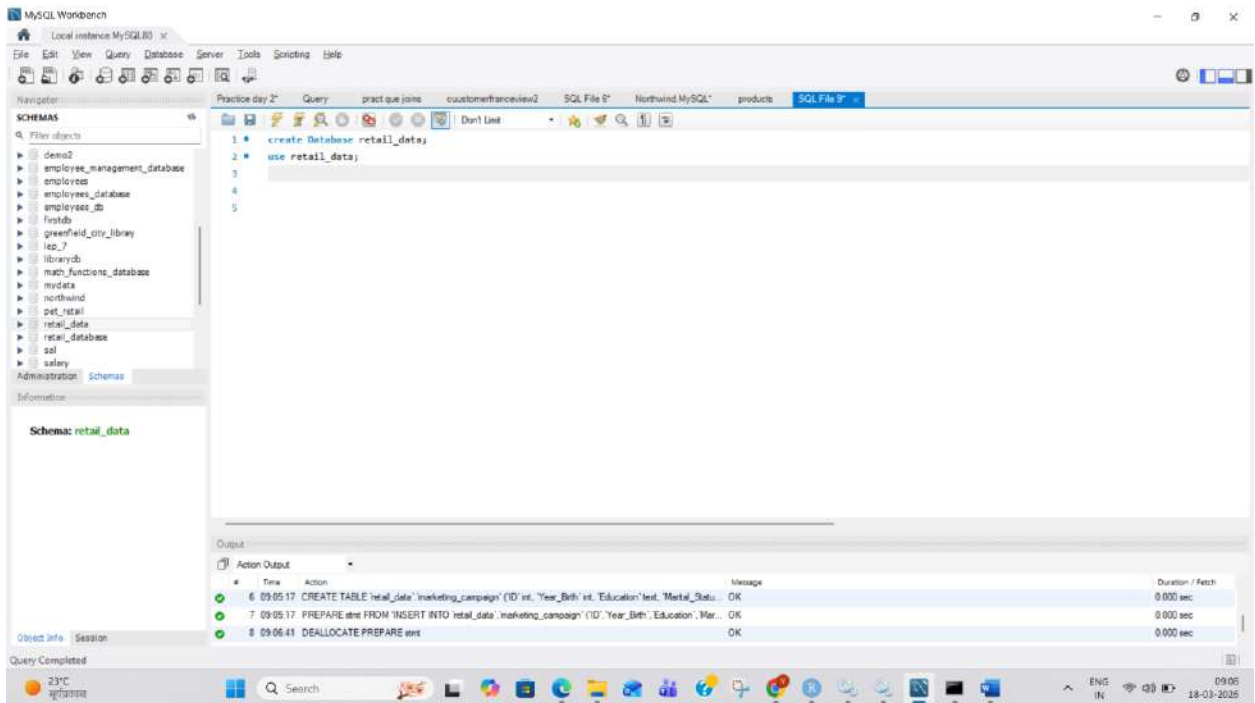
Response "1" is consistently low across all marital statuses — the orange segment is always a small fraction of the blue.

Married people have the highest "1" responses (98), followed by Single (106) — Single has a *higher "1" rate* relative to its size than Married.

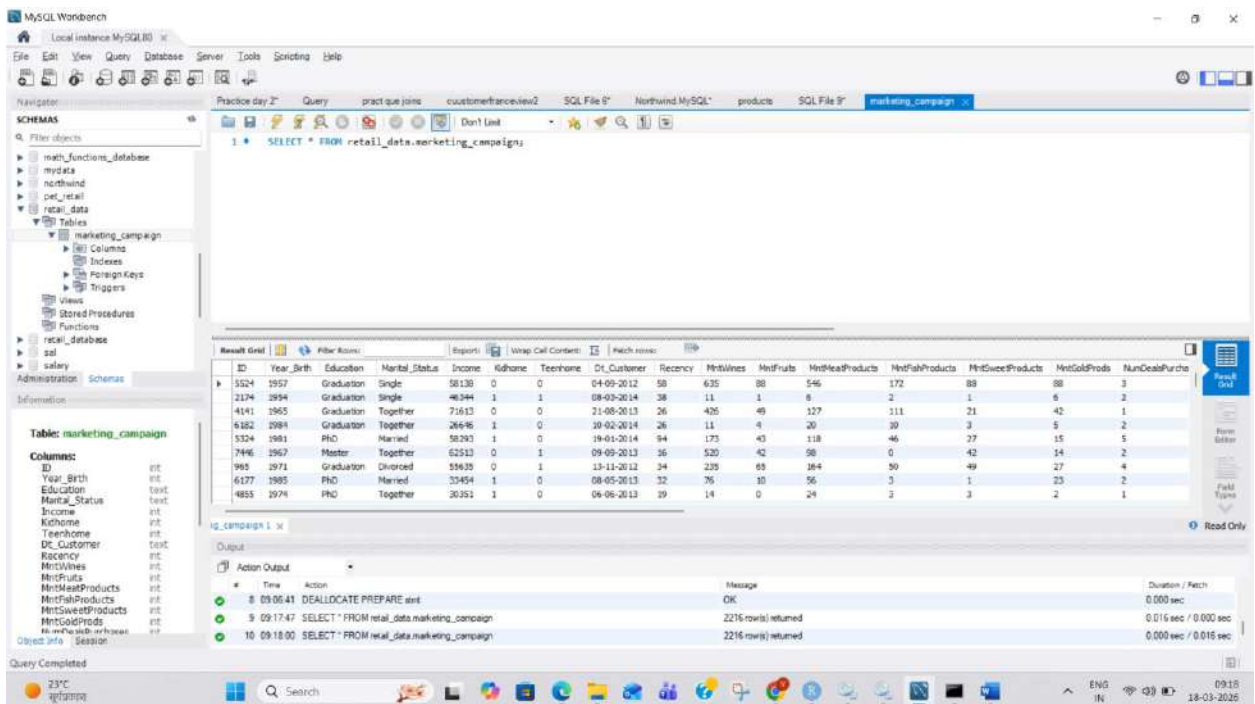
SQL Tasks:

2. Data Loading:

- Create a schema named "retail_data"
- Set "retail_data" as the default schema



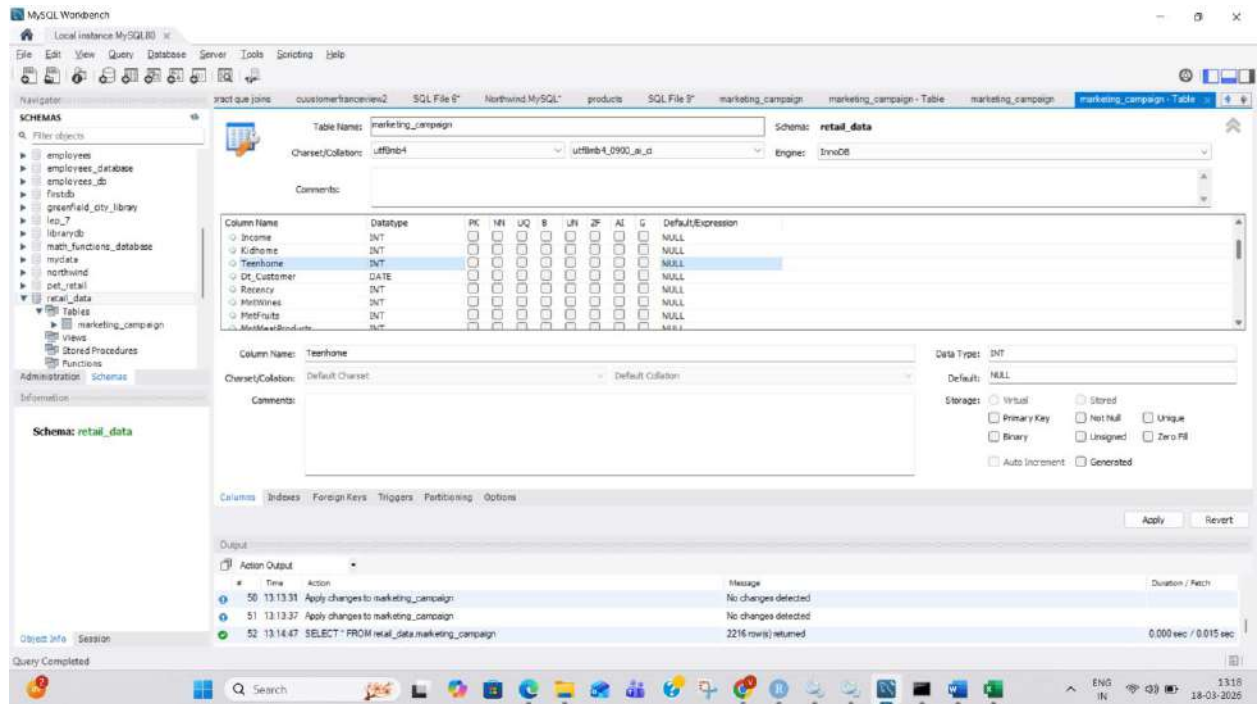
- Create tables to store the retail transaction data



- Set ct_customer as the datetime field while loading the data and apply the appropriate date format

First changes format in excel to yyyy-mm-dd hh:mm:ss

Then load data in workbench and then change datatype of Dt_Customers to date type



3. Data Preprocessing:

- Calculate the total number of customer encounters in the marketing campaign dataset

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL code:

```

1. create Database retail_data;
2. use retail_data;
3.
4. select count(*) as Total_encounters
5. from marketing_campaign;
6.

```

The 'Result Grid' shows the output of the query:

Table: marketing_campaign
Total_encounters
2216

The 'Output' tab shows the execution log:

Time	Action	Message	Duration / Patch
00 13:43:20	create Database retail_data	Error Code: 1007 Can't create database 'retail_data': database exists	0.000 sec
01 13:44:51	select count(*) as Total_encounters from marketing_campaign	7 row(s) returned	0.031 sec / 0.000 sec

Total Encounters:2216

- Identify the top 10 most purchased products in the dataset, such as Wines, Meat Products, etc.

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL code:

```

5. from marketing_campaign;
6.
7.
8. SELECT
9.     CASE
10.        WHEN product_name = 'HotWines' THEN 'Wines'
11.        WHEN product_name = 'HotFruits' THEN 'Fruits'
12.        WHEN product_name = 'HotMeatProducts' THEN 'Meat Products'
13.        WHEN product_name = 'HotFishProducts' THEN 'Fish Products'
14.        WHEN product_name = 'HotSweetProducts' THEN 'Sweet Products'
15.        WHEN product_name = 'HotGoldProducts' THEN 'Gold Products'
16.    END AS product_category,
17.    sum(product_amount) AS total_purchases

```

The 'Result Grid' shows the output of the query:

product_category	total_purchases
Wines	670283
MeatProducts	370063
GoldProducts	91427
SweetProducts	99906
Fruits	58405

The 'Output' tab shows the execution log:

Time	Action	Message	Duration / Patch
54 13:37:40	create Database retail_data	Error Code: 1007 Can't create database 'retail_data': database exists	0.000 sec
55 13:37:40	create Database retail_data	Error Code: 1007 Can't create database 'retail_data': database exists	0.000 sec
56 13:38:13	create Database retail_data	Error Code: 1007 Can't create database 'retail_data': database exists	0.000 sec

- Find the count of response values

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'marketing_campaign' table. The main query editor contains the following SQL code:

```

1  marketing_campaign UNION ALL SELECT
2  'MintGoldProds' AS product_name,
3  'MintGoldProds' AS product_amount
4  FROM
5  marketing_campaign) AS subquery
6  GROUP BY product_category
7  ORDER BY total_purchases DESC
8  LIMIT 10;
9
10 select response ,count(*) from marketing_campaign
11 group by response;

```

The 'Result Grid' shows the output of the query:

response	count(*)
1	333
0	1882

The 'Output' pane shows the execution details of the query, including the time taken and the number of rows returned.

1-> 333

0-> 1882

- Determine the distribution of customers based on their education level and marital status

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'marketing_campaign' table. The main query editor contains the following SQL code:

```

44 select response ,count(*) from marketing_campaign
45 group by response;
46
47
48 SELECT
49 Education, Marital_Status, COUNT(*) AS Customer_count
50 FROM
51 marketing_campaign
52 GROUP BY Education , Marital_Status
53 order by Education , Marital_Status;
54
55

```

The 'Result Grid' shows the output of the query:

Education	Marital_Status	Customer_count
2n Cycle	Divorced	23
2n Cycle	Married	89
2n Cycle	Single	36
2n Cycle	Together	56
2n Cycle	Widow	9
Basic	Divorced	1
Basic	Married	20
Basic	Widow	18

The 'Output' pane shows the execution details of the query, including the time taken and the number of rows returned.

- Identify the average income of customers who participated in the marketing campaign

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'retail_data' database selected. The 'Query' editor in the center contains the following SQL query:

```

46 group by response;
47
48 * SELECT
49     Education, Marital_Status, COUNT(*) AS Customer_count
50 FROM
51     marketing_campaign
52 GROUP BY Education, Marital_Status
53 order by Education, Marital_Status;
54
55 * select avg(Income) as Avg_Income
56 from marketing_campaign;

```

The 'Result Grid' at the bottom shows the results of the query:

Avg_Income
52247.2514

The 'Action Output' pane at the bottom right shows the execution details of the query, including the time taken and the number of rows returned.

Avg income = 52,247.25 rs

- Calculate the total number of promotions accepted by customers in each campaign

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'retail_data' database selected. The 'Query' editor in the center contains the following SQL query:

```

53 order by Education, Marital_Status;
54
55 * select avg(Income) as Avg_Income
56 from marketing_campaign;
57
58 * SELECT
59     Id,
60     SUM(AcceptedCmp3 + AcceptedCmp4 + AcceptedCmp5 + AcceptedCmp1 + AcceptedCmp2) AS Total_Promotions_accepted
61 FROM
62     marketing_campaign
63 GROUP BY Id;
64

```

The 'Result Grid' at the bottom shows the results of the query:

Id	Total_Promotions_accepted
5324	0
2124	0
4141	0
6182	0
5324	0
7446	0
965	0
4122	0

The 'Action Output' pane at the bottom right shows the execution details of the query, including the time taken and the number of rows returned.

- Identify the distribution of customers' responses to the last campaign

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists various databases, with 'retail_data' expanded to show the 'marketing_campaign' table. The 'Query' editor contains the following SQL code:

```

61 FROM
62 marketing_campaign
63 GROUP BY id;
64
65 SELECT
66 response, COUNT(*) AS response_count
67 FROM
68 marketing_campaign
69 GROUP BY response;
70
71

```

The 'Result Grid' shows the results of the second query, displaying two rows of data:

response	response_count
1	533
0	1883

The 'Output' pane at the bottom shows the execution progress of the queries, including the time taken and the number of rows returned.

- Calculate the average number of children and teenagers in customers' households

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists various databases, with 'retail_data' expanded to show the 'marketing_campaign' table. The 'Query' editor contains the following SQL code:

```

64
65 SELECT
66 response, COUNT(*) AS response_count
67 FROM
68 marketing_campaign
69 GROUP BY response;
70
71 SELECT
72 AVG(kidhome) AS Avg_Children, AVG(teenhome) AS Avg_Teenagers
73 FROM
74 marketing_campaign;

```

The 'Result Grid' shows the results of the second query, displaying two rows of data:

Avg_Children	Avg_Teenagers
0.4438	0.5054

The 'Output' pane at the bottom shows the execution progress of the queries, including the time taken and the number of rows returned.

Avg of children I 0.44 and teenagers is 0.50

- Create an Age column by subtracting year_birth from the current year

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'retail_data' database selected, with the 'marketing_campaign' table highlighted. The 'Table: marketing_campaign' pane shows the following columns: ID (int), Year_Birth (int), Education (text), Marital_Status (text), Income (int), Kidhome (int), Teenhome (int), DC_Customer (date), Rascency (int), MntWines (int), MntFruits (int), MntMeatProducts (int), MntFishProducts (int), MntSweetProducts (int), MntGoldProds (int), and MntWinech (int). The main query editor shows the following SQL code:

```

64
65 SELECT
66     response, COUNT(*) AS response_count
67 FROM
68     marketing_campaign
69 GROUP BY response;
70
71 SELECT
72     AVG(Kidhome) AS Avg_Children, AVG(Teenhome) AS Avg_Teenagers
73 FROM
74     marketing_campaign;
75
76 alter table marketing_campaign
77 add Age1 int;
78
79 set sql_safe_updates = 0;
80 UPDATE marketing_campaign
81 SET
82     Age1 = YEAR(NOW()) - year_birth;
83 set sql_safe_updates = 1;
84

```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Patch
74	18.06.25	set sql_safe_updates = 0	0 row(s) affected	0.000 sec
75	18.06.25	UPDATE marketing_campaign SET Age1 = YEAR(NOW()) - year_birth	2216 row(s) affected Rows matched: 2216 Changed: 2216 Warnings: 0	0.500 sec
76	18.06.27	set sql_safe_updates = 1	0 row(s) affected	0.000 sec

Query Completed

- Create Age_group columns based on the below condition: WHEN Age BETWEEN 18 AND 25 THEN '18-25' WHEN Age BETWEEN 26 AND 35 THEN '26-35' WHEN Age BETWEEN 36 AND 45 THEN '36-45' WHEN Age BETWEEN 46 AND 55 THEN '46-55' ELSE '56+'

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'retail_data' database selected, with the 'marketing_campaign' table highlighted. The 'Table: marketing_campaign' pane shows the following columns: ID (int), Year_Birth (int), Education (text), Marital_Status (text), Income (int), Kidhome (int), Teenhome (int), DC_Customer (date), Rascency (int), MntWines (int), MntFruits (int), MntMeatProducts (int), MntFishProducts (int), MntSweetProducts (int), MntGoldProds (int), and MntWinech (int). The main query editor shows the following SQL code:

```

79 set sql_safe_updates = 0;
80 UPDATE marketing_campaign
81 SET
82     Age1 = YEAR(NOW()) - year_birth;
83 set sql_safe_updates = 1;
84
85 alter table marketing_campaign
86 add Age_group_1 text;
87
88 set sql_safe_updates = 0;
89 UPDATE marketing_campaign
90 SET
91     Age_group = CASE
92         WHEN Age BETWEEN 18 AND 25 THEN '18-25'
93         WHEN Age BETWEEN 26 AND 35 THEN '26-35'
94         WHEN Age BETWEEN 36 AND 45 THEN '36-45'
95         WHEN Age BETWEEN 46 AND 55 THEN '46-55'
96         ELSE '56+'
97     END;
98 set sql_safe_updates = 1;
99

```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Patch
83	18.18.15	set sql_safe_updates = 0	0 row(s) affected	0.000 sec
84	18.18.15	UPDATE marketing_campaign SET Age_group = CASE WHEN Age BETWEEN 18 AND 25 TH...	0 row(s) affected Rows matched: 2216 Changed: 0 Warnings: 0	0.015 sec
85	18.18.15	set sql_safe_updates = 1	0 row(s) affected	0.015 sec

Query Completed

- Determine the average number of visits per month for customers in each age group

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

employees employees_database employees_db firstds greenfield_city_library lep_7 librarydb math_functions_database mydata northwind pet_retail retail_data

Tables

marketing_campaign

Views

Stored Procedures

Functions

Administration Schemas

Information

Table: marketing_campaign

Columns:

ID int
Year_Birth int
Education text
Marital_Status int
Income int
Kidhome int
Teenhome int
DC_Customer date
Raceny int
MntWines int
MntFruits int
MntMeatProducts int
MntFishProducts int
MntSweetProducts int
MntGoldProds int
MntWines int
MntFruits int

Query

```

100 * Alter table marketing_campaign
101 drop column Age_group ;
102
103 alter table marketing_campaign
104 add Age_group_1 text;
105
106 set sql_safe_updates = 0;
107 UPDATE marketing_campaign
108 SET
109     Age_group_1 = CASE
110         WHEN Age BETWEEN 18 AND 35 THEN '18-35'
111         WHEN Age BETWEEN 36 AND 45 THEN '36-45'
112         WHEN Age BETWEEN 46 AND 55 THEN '46-55'
113         ELSE '56+'
114     END;
115
116 set sql_safe_updates = 1;
117 * select * from marketing_campaign;
118
119

```

Output

Action Output

#	Time	Action	Message	Duration / Patch
96	18.33.19	SELECT * FROM retail_data.marketing_campaign	2216 rows returned	0.016 sec / 0.015 sec
97	18.34.42	Alter table marketing_campaign drop column Age_group	0 rows affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
98	18.34.53	SELECT * FROM retail_data.marketing_campaign	2216 rows returned	0.000 sec / 0.032 sec

Query Completed

33°C

18:35 18-03-2026

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

employees employees_database employees_db firstds greenfield_city_library lep_7 librarydb math_functions_database mydata northwind pet_retail retail_data

Tables

marketing_campaign

Views

Stored Procedures

Functions

Administration Schemas

Information

Table: marketing_campaign

Columns:

ID int
Year_Birth int
Education text
Marital_Status int
Income int
Kidhome int
Teenhome int
DC_Customer date
Raceny int
MntWines int
MntFruits int
MntMeatProducts int
MntFishProducts int
MntSweetProducts int
MntGoldProds int
MntWines int
MntFruits int

Query

```

1 * SELECT * FROM retail_data.marketing_campaign;

```

Result Grid

Purchases	NumStorePurchases	NumWebVisitsNorth	AcceptedCmp3	AcceptedCmp4	AcceptedCmp5	AcceptedCmp1	AcceptedCmp2	Complan	Z_CostContact	Z_Revenue	Response	Age	Age1	Age_group_1
4	7	0	0	0	0	0	0	0	3	11	1	67	63	56+
2	5	0	0	0	0	0	0	0	3	11	0	70	72	56+
10	4	0	0	0	0	0	0	0	3	11	0	59	61	56+
6	6	0	0	0	0	0	0	0	3	11	0	40	42	36-45
6	5	0	0	0	0	0	0	0	3	11	0	43	45	36-45
10	6	0	0	0	0	0	0	0	3	11	0	57	59	56+
7	6	0	0	0	0	0	0	0	3	11	0	53	55	46-55
4	8	0	0	0	0	0	0	0	3	11	0	39	41	36-45
2	9	0	0	0	0	0	0	0	3	11	1	50	52	46-55
6	20	1	0	0	0	0	0	0	3	11	0	74	76	56+
3	8	0	0	0	0	0	0	0	3	11	0	48	50	46-55

Output

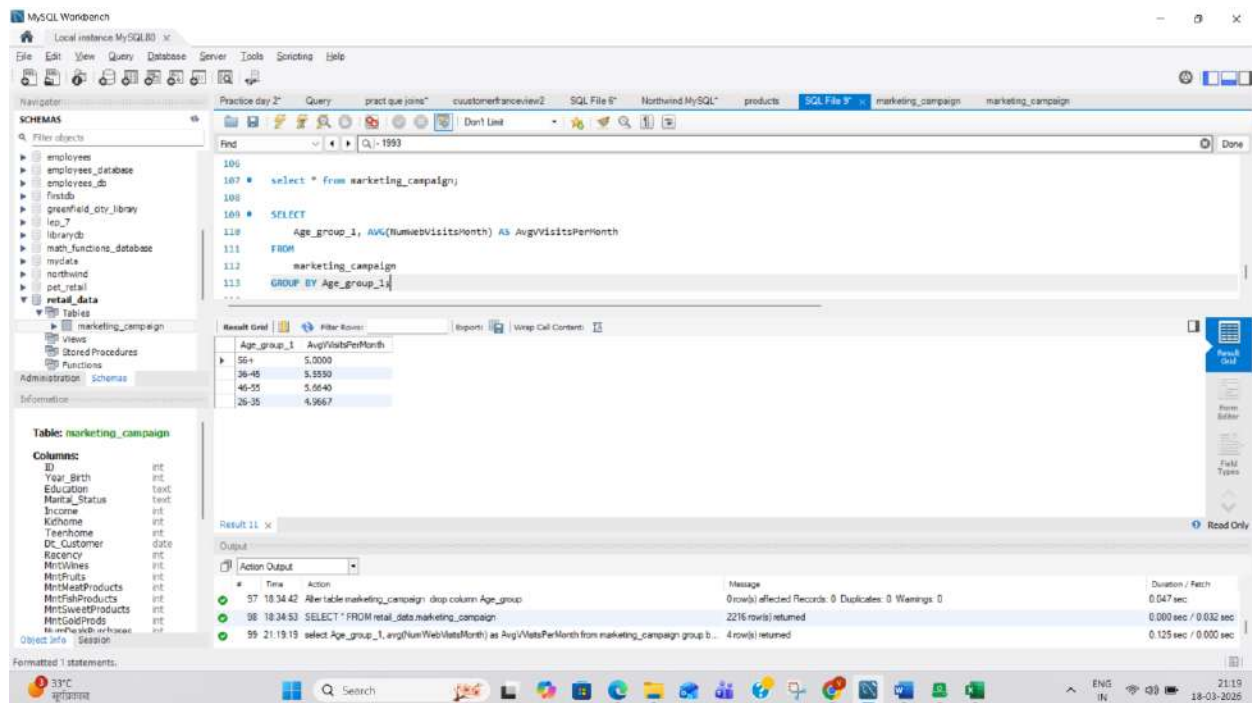
Action Output

#	Time	Action	Message	Duration / Patch
96	18.33.19	SELECT * FROM retail_data.marketing_campaign	2216 rows returned	0.016 sec / 0.015 sec
97	18.34.42	Alter table marketing_campaign drop column Age_group	0 rows affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
98	18.34.53	SELECT * FROM retail_data.marketing_campaign	2216 rows returned	0.000 sec / 0.032 sec

Query Completed

33°C

18:35 18-03-2026

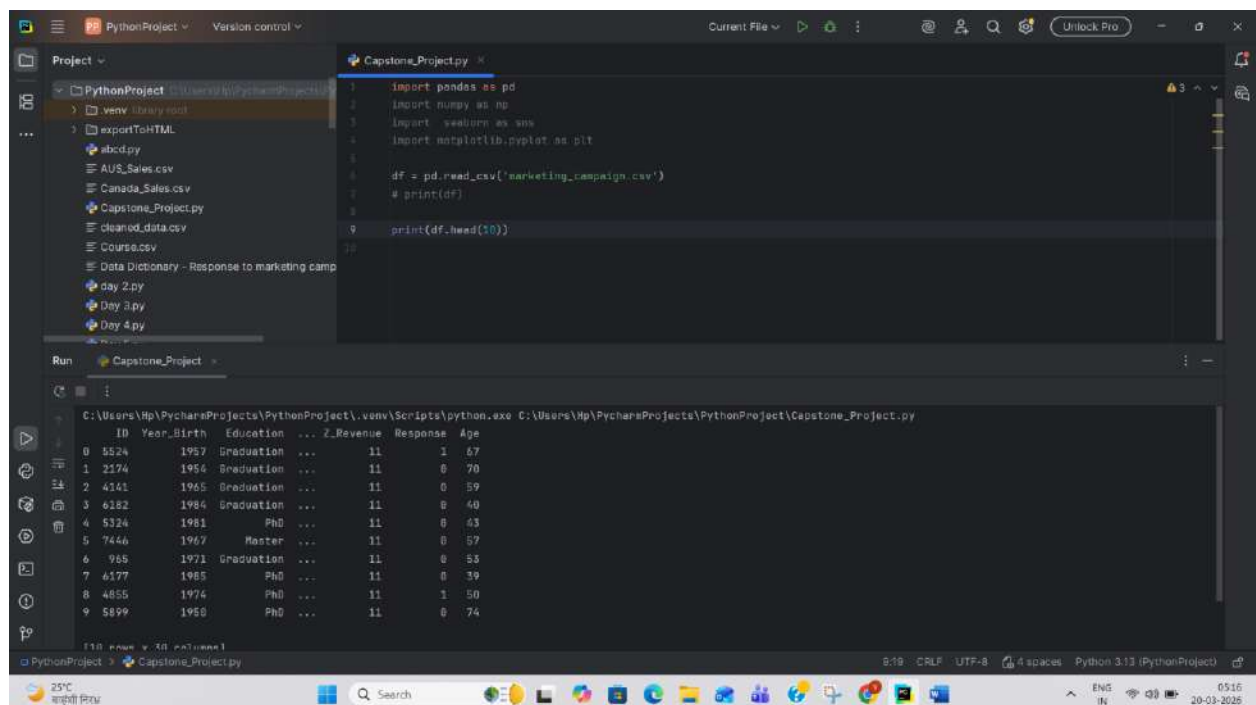


Python Tasks:

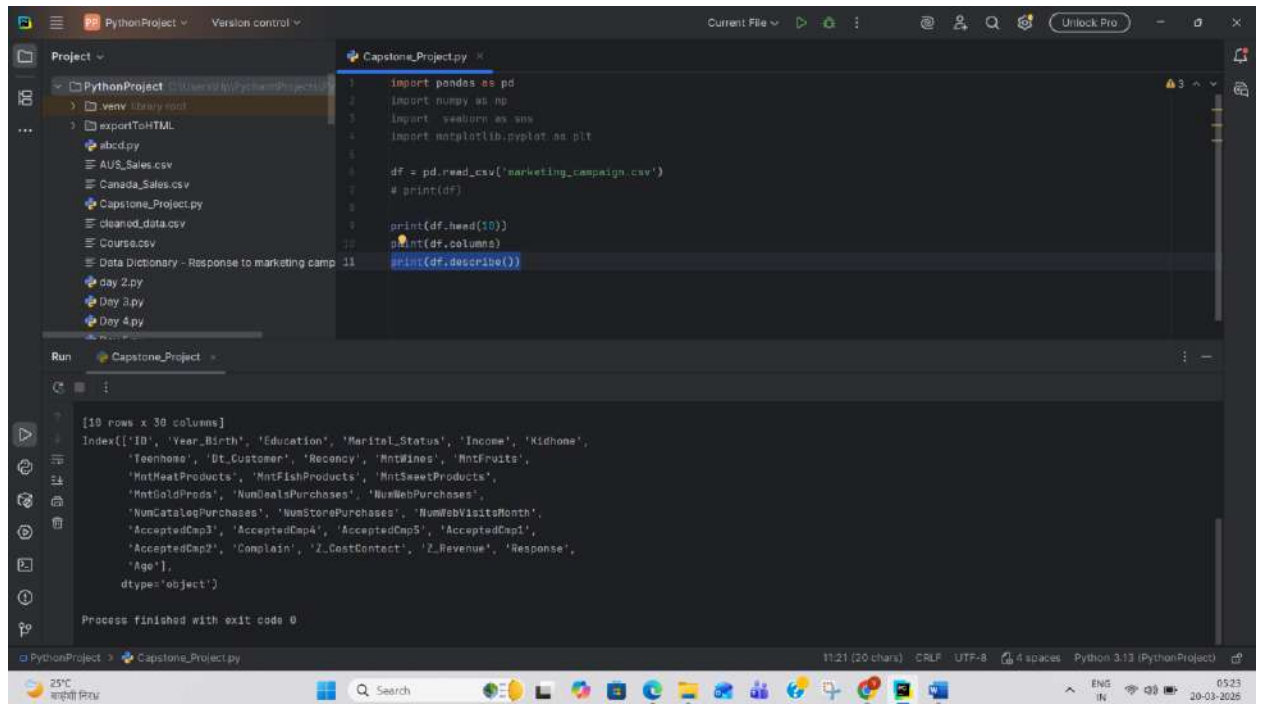
4 Exploratory Data Analysis

Task 1: General Data Overview :

- Check the first few rows of the dataset to understand its structure



b. Check the data types of each column



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'Capstone_Project.py' with several files including 'AUS_Sales.csv', 'Canada_Sales.csv', 'cleaned_data.csv', 'Course.csv', and 'Data Dictionary - Response to marketing camp'. The code editor shows the following code:

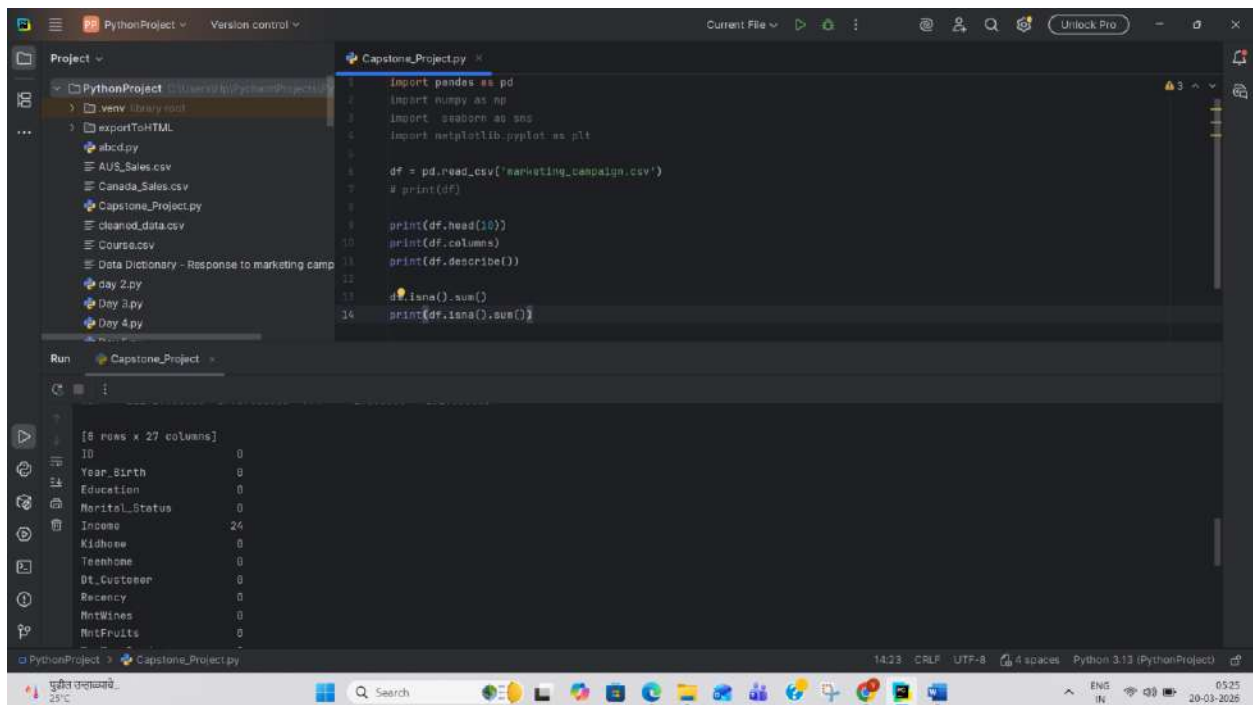
```
1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv('marketing_campaign.csv')
7 # print(df)
8
9 print(df.head(10))
10 print(df.columns)
11 print(df.describe())
```

The output of the code is displayed in the console below the code editor. It shows the first 10 rows of the dataset and the data types of each column. The output is as follows:

```
[10 rows x 30 columns]
Index(['ID', 'Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome',
       'Teenhome', 'Dt_Customer', 'Recency', 'MntWines', 'MntFruits',
       'MntMeatProducts', 'MntFishProducts', 'MntSeafoodProducts',
       'MntGoldProds', 'NumDealsPurchases', 'NumWebPurchases',
       'NumCatalogPurchases', 'NumStorePurchases', 'NumVisitsMonth',
       'AcceptedCap3', 'AcceptedCap4', 'AcceptedCap5', 'AcceptedCap1',
       'AcceptedCap2', 'Complain', 'Z_CostContact', 'Z_Revenue', 'Response',
       'Age'],
      dtype='object')
```

The process finished with exit code 0.

c. Check for any missing values in the dataset



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'Capstone_Project.py' with several files including 'AUS_Sales.csv', 'Canada_Sales.csv', 'cleaned_data.csv', 'Course.csv', and 'Data Dictionary - Response to marketing camp'. The code editor shows the following code:

```
1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv('marketing_campaign.csv')
7 # print(df)
8
9 print(df.head(10))
10 print(df.columns)
11 print(df.describe())
12
13 df.isna().sum()
14 print(df.isna().sum())
```

The output of the code is displayed in the console below the code editor. It shows the first 10 rows of the dataset and the data types of each column. The output is as follows:

```
[8 rows x 27 columns]
ID          0
Year_Birth  0
Education   0
Marital_Status  0
Income      24
Kidhome     0
Teenhome    0
Dt_Customer  0
Recency      0
MntWines    0
MntFruits   0
MntMeatProducts  0
MntFishProducts  0
MntSeafoodProducts  0
MntGoldProds    0
NumDealsPurchases  0
NumWebPurchases   0
NumCatalogPurchases  0
NumStorePurchases  0
NumVisitsMonth     0
AcceptedCap3       0
AcceptedCap4       0
AcceptedCap5       0
AcceptedCap1       0
AcceptedCap2       0
Complain          0
Z_CostContact     0
Z_Revenue         0
Response          0
Age              0
```

The process finished with exit code 0.

Observations

a. Dataset Structure:

- The dataset comprises several columns representing various features related to marketing campaigns.
- We can observe the structure and format of the data by examining the first few rows of the dataset.

b. Data Types:

- Each column has a specific data type, which provides insights into the nature of the features present in the dataset.

c. Missing Values Handling:

- There were missing values in the 'Income' column, which were handled by replacing them with the median income value.
- After handling missing values in the 'Income' column, there are no other missing values present in the dataset.

The screenshot shows a Python IDE with a project named 'PythonProject'. The file explorer on the left lists several files, including 'Capstone_Project.py'. The main editor displays the code for 'Capstone_Project.py', which reads a CSV file named 'marketing_campaign.csv', prints the first 10 rows, and calculates the median income. The median income is 51381.5, and this value is used to replace missing values in the 'Income' column. The output console at the bottom shows the command executed and the resulting median income value.

```

1 df = pd.read_csv('marketing_campaign.csv')
2 # print(df)
3
4 # print(df.head(10))
5 # print(df.columns)
6 # print(df.describe())
7
8 # df.info()
9 # print(df.info())
10
11 median_income = df['Income'].median()
12 df['Income'] = df['Income'].fillna(median_income)
13 print(median_income)
14

```

Run: Capstone_Project

C:\Users\Hp\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Hp\PycharmProjects\PythonProject\Capstone_Project.py

51381.5

Process finished with exit code 0

median = 51381.5

The screenshot shows a Jupyter Notebook with the following code:

```

1 df = pd.read_csv('marketing_campaign.csv')
2 # print(df)
3
4 # print(df.head(10))
5 # print(df.columns)
6 # print(df.describe())
7
8 # df.isna().sum()
9 # print(df.isna().sum())
10
11 median_income = df['Income'].median()
12 df['Income'] = df['Income'].fillna(median_income)
13 print(median_income)
14 print(df.isna().sum())

```

The output shows the first few rows of the dataset:

ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	MntFruits	MntMassPremiums
51581.5	0	0	0	0	0	0	0	0	0	0	0

Missing values filled with median income

Task 2: Descriptive Statistics:

- Compute summary statistics for numerical columns (mean, median, min, max, and standard deviation)

The screenshot shows the same Jupyter Notebook with additional code for computing summary statistics:

```

15 # df.isna().sum()
16 # print(df.isna().sum())
17
18 median_income = df['Income'].median()
19 df['Income'] = df['Income'].fillna(median_income)
20 # print(median_income)
21 # print(df.isna().sum())
22 # print(df)
23
24 Summary_stats = (df.describe())
25 print("Summary Statistics for Numerical columns:")
26 print(Summary_stats)

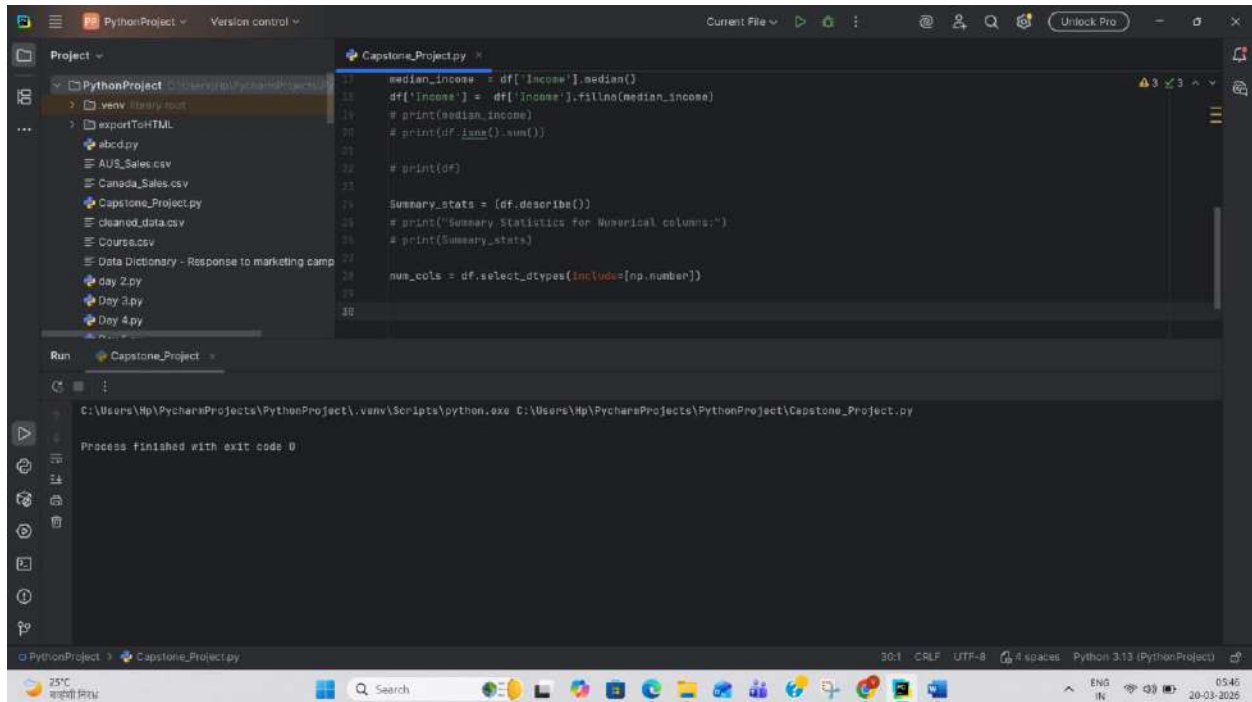
```

The output shows the summary statistics for numerical columns:

	ID	Year_Birth	...	Response	Age
count	2240.000000	2240.000000	...	2240.000000	2240.000000
mean	5592.159821	1968.805804	...	0.149107	55.194196
std	3246.642198	11.984069	...	0.354274	11.984069
min	0.000000	1893.000000	...	0.000000	28.000000
25%	2828.250000	1959.000000	...	0.000000	47.000000
50%	5458.500000	1978.000000	...	0.000000	54.000000
75%	8427.750000	1977.000000	...	0.000000	65.000000
max	11191.000000	1996.000000	...	1.000000	131.000000

b. Explore the distribution of numerical variables using histograms or box plots

First we have define numerical columns then we will plot charts



```
17 median_income = df['Income'].median()
18 df['Income'] = df['Income'].fillna(median_income)
19 # print(median_income)
20 # print(df['Income'].sum())
21
22 # print(df)
23
24 Summary_stats = (df.describe())
25 # print('Summary Statistics for Numerical columns:')
26 # print(Summary_stats)
27
28 num_cols = df.select_dtypes(include=[np.number])
29
30
```

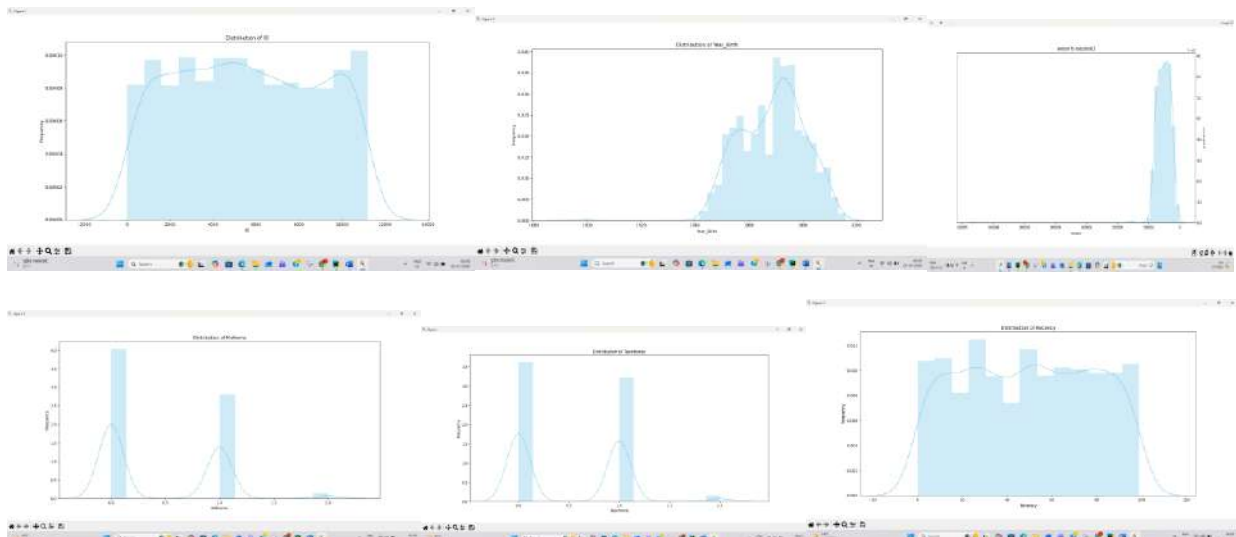
Run Capstone_Project

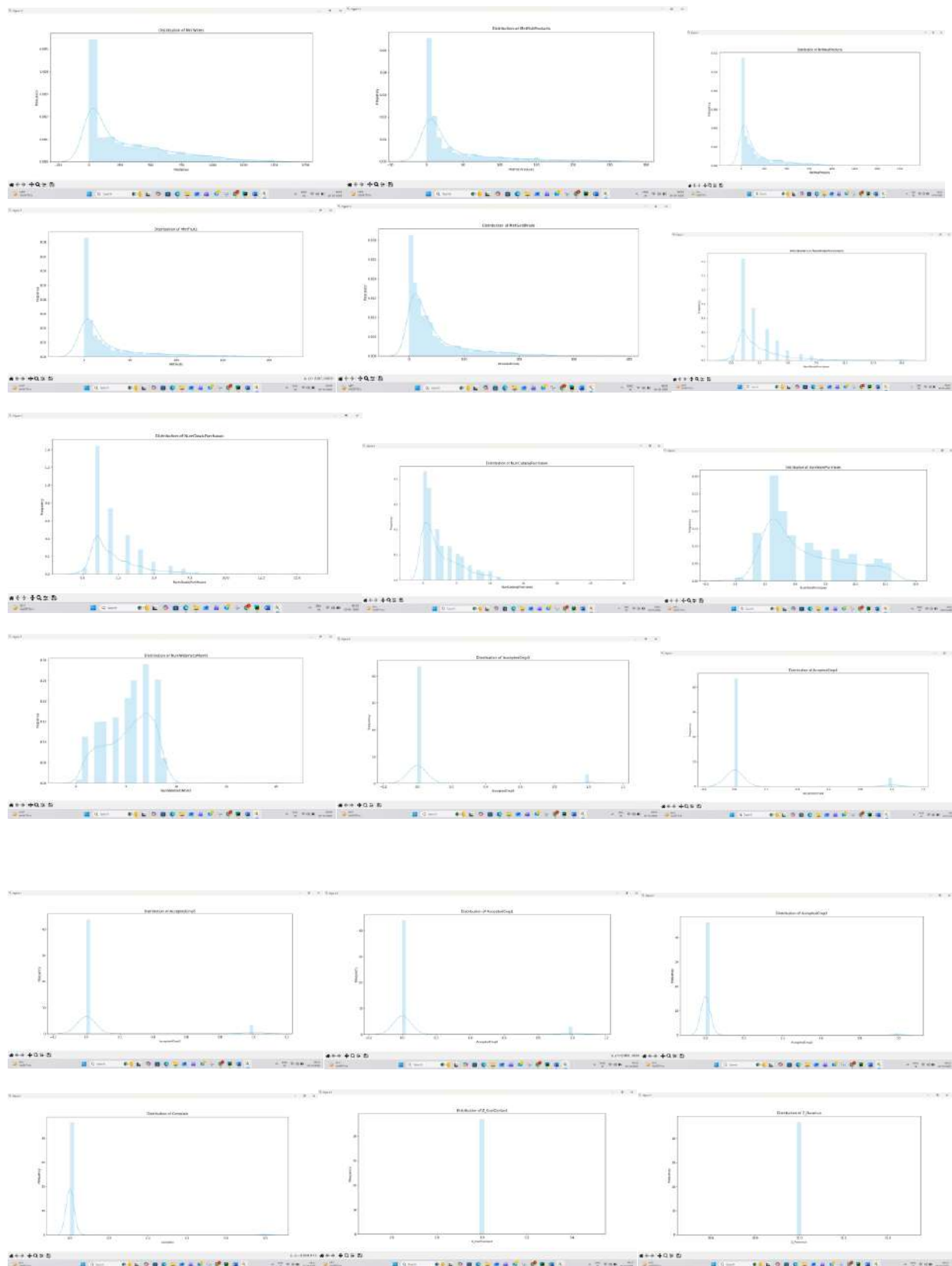
C:\Users\Hp\PchamProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Hp\PchamProjects\PythonProject\Capstone_Project.py

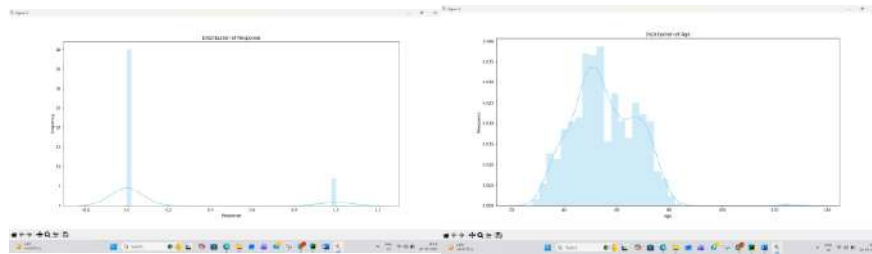
Process finished with exit code 0

Numerical variables using histogram

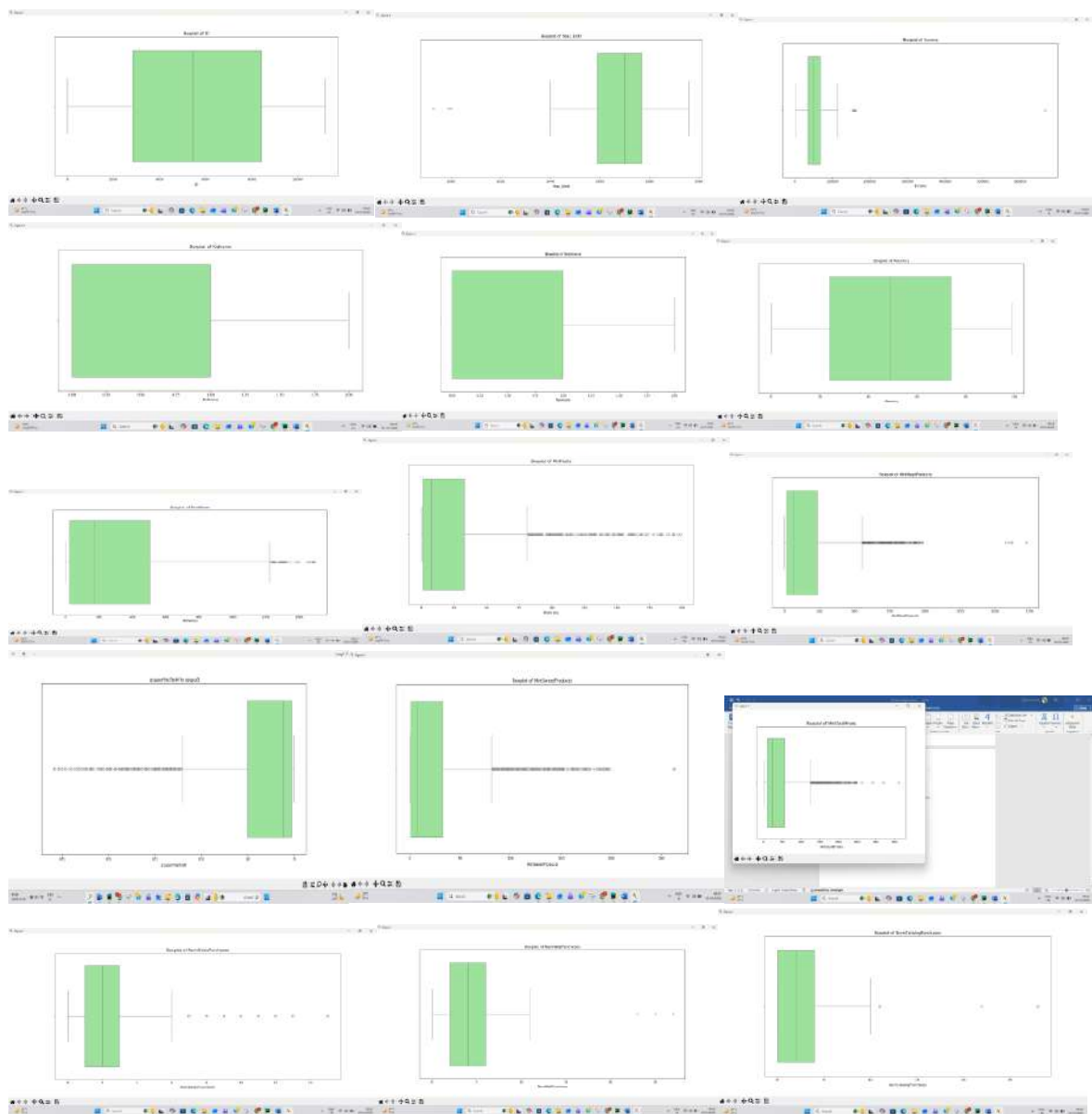
It will show the charts for each numerical variables in data

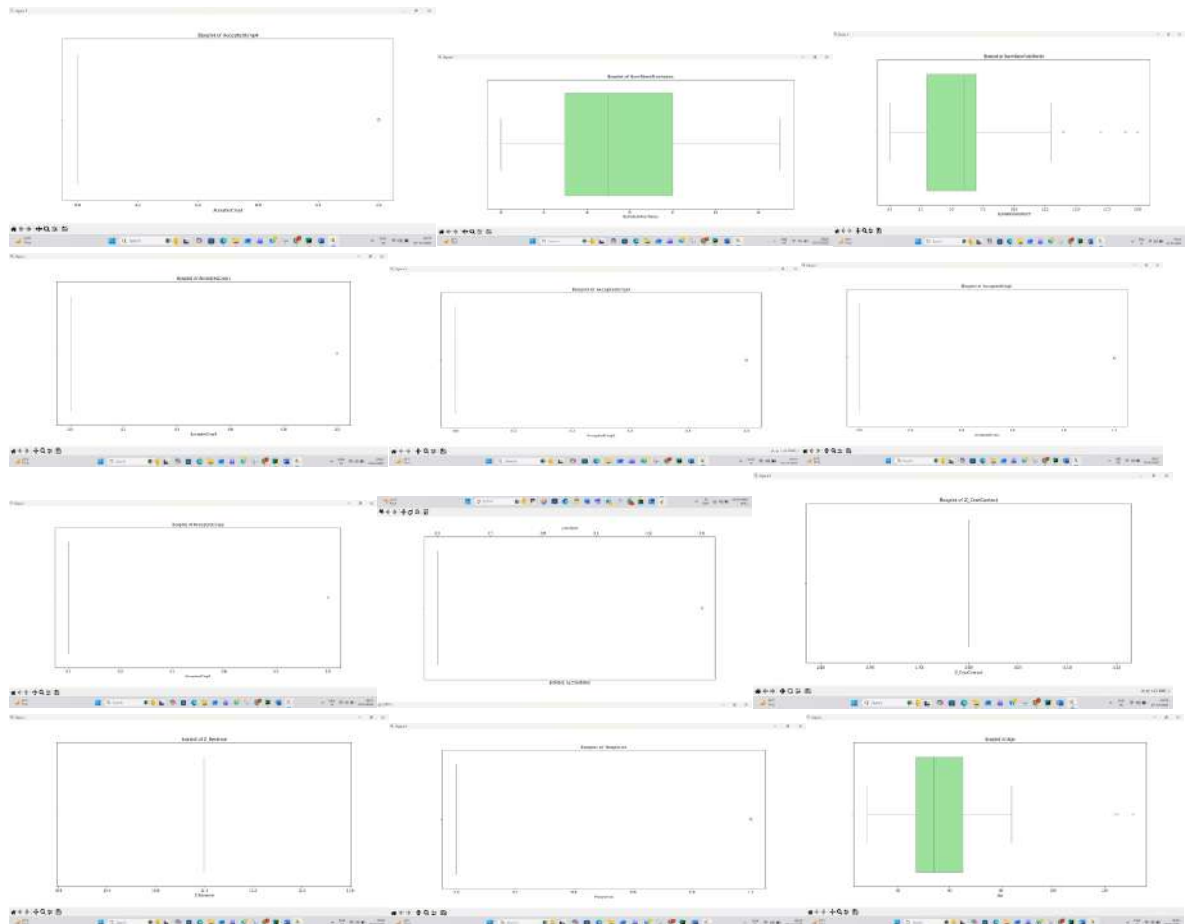






Numerical plot using Box Plot





. Summary Statistics:

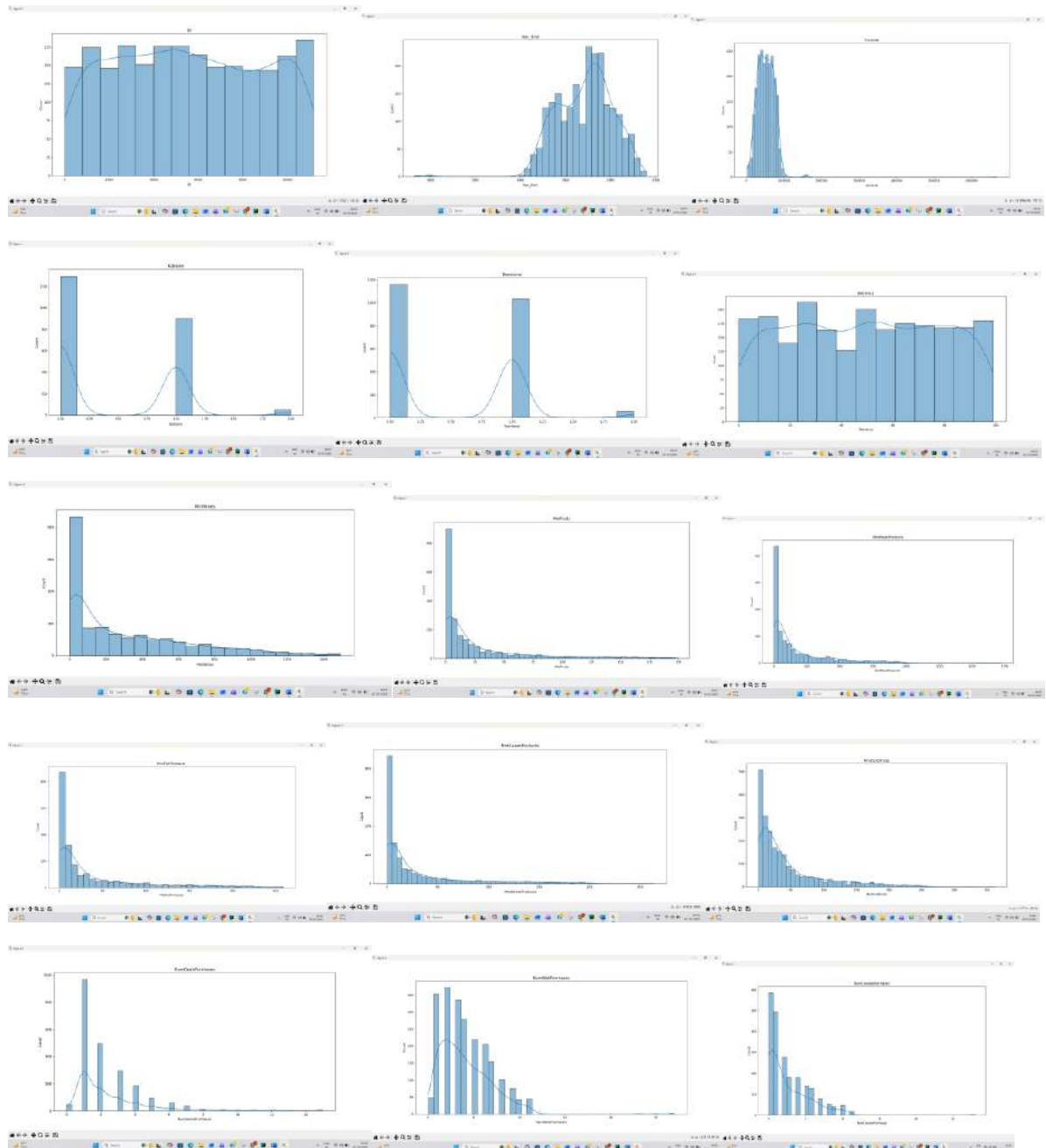
- Summary statistics including mean, median, minimum, maximum, and standard deviation were computed for numerical columns, providing an overview of their distribution.

b. Distribution of Numerical Variables:

- The distribution of numerical variables was explored using histograms, revealing insights into the spread and central tendency of the data.

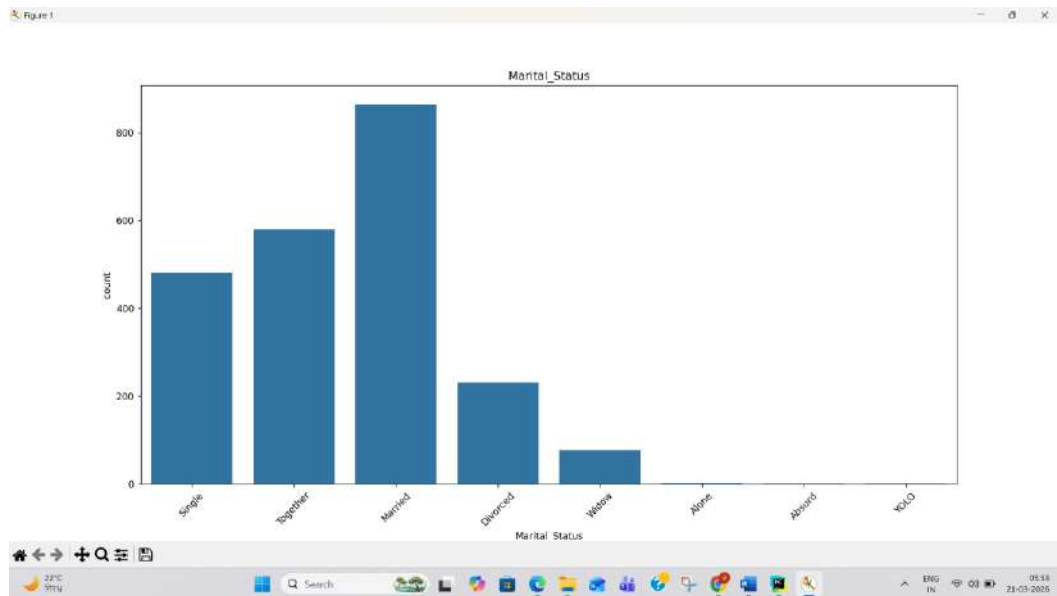
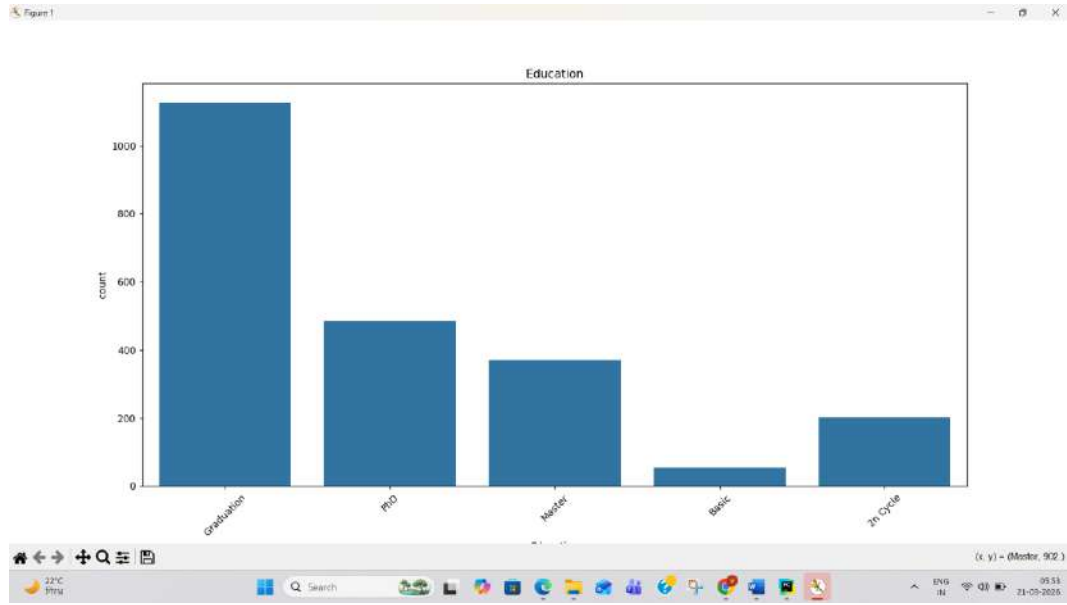
Task 3: Univariate Analysis:

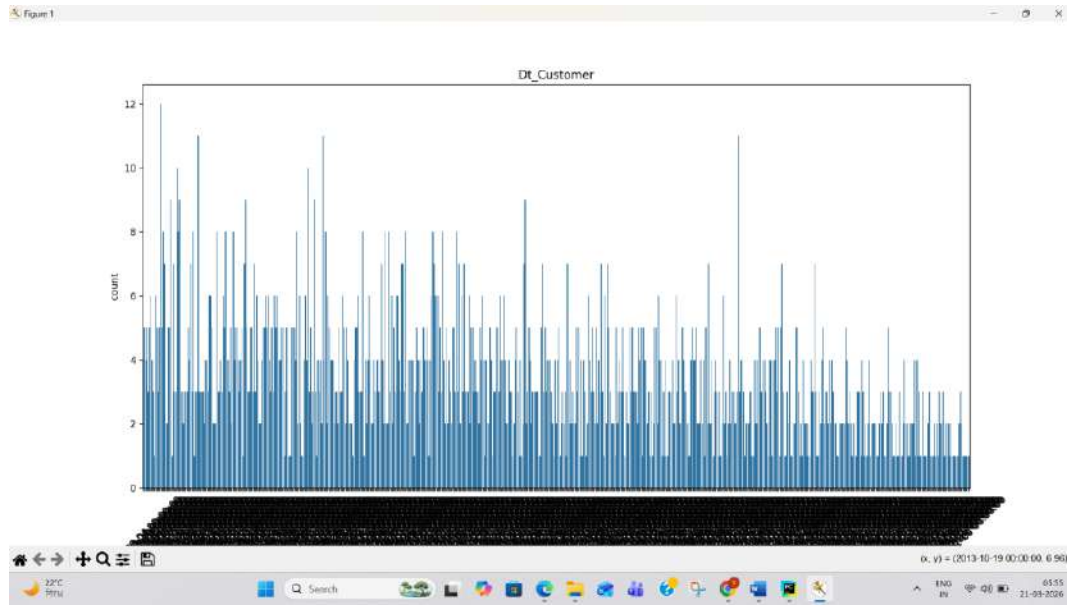
- Explore the distribution of each numerical variable using histograms or kernel density plots





b. Explore the distribution of each categorical variable using bar plots or pie charts





a. Distribution of Numerical Variables:

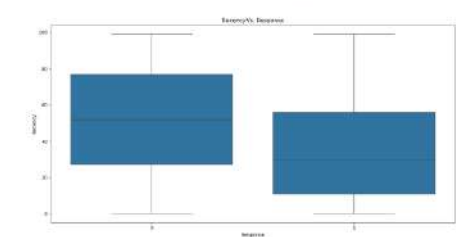
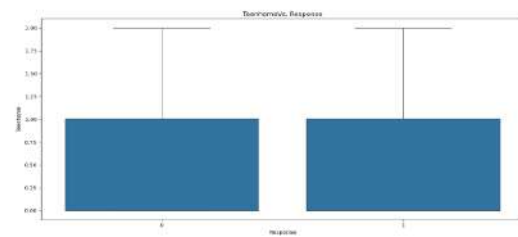
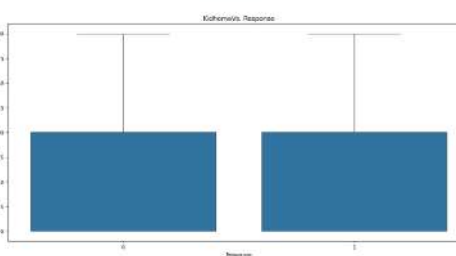
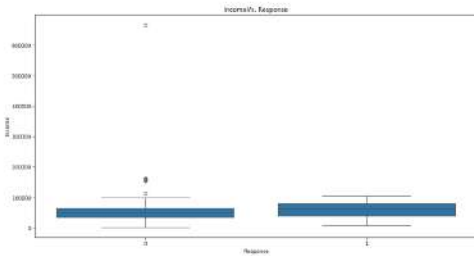
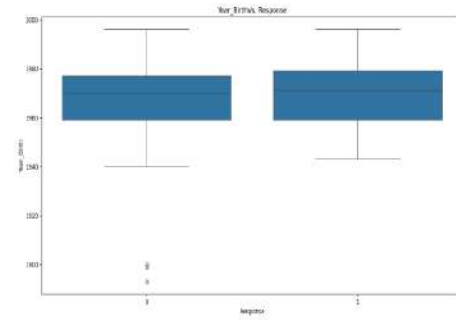
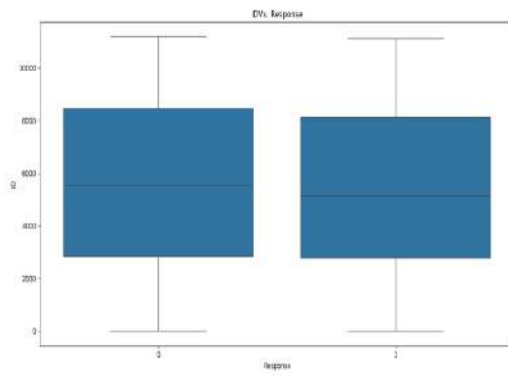
- Histograms plots were utilized to visualize the distribution of each numerical variable, enabling a better understanding of their spread and shape.

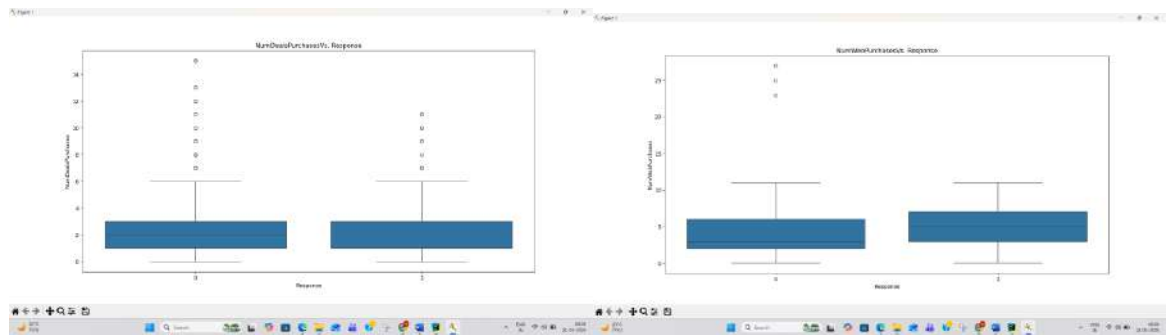
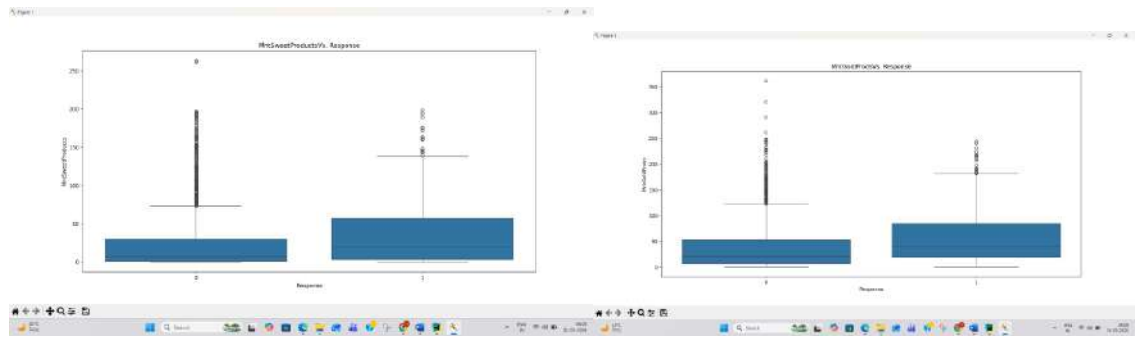
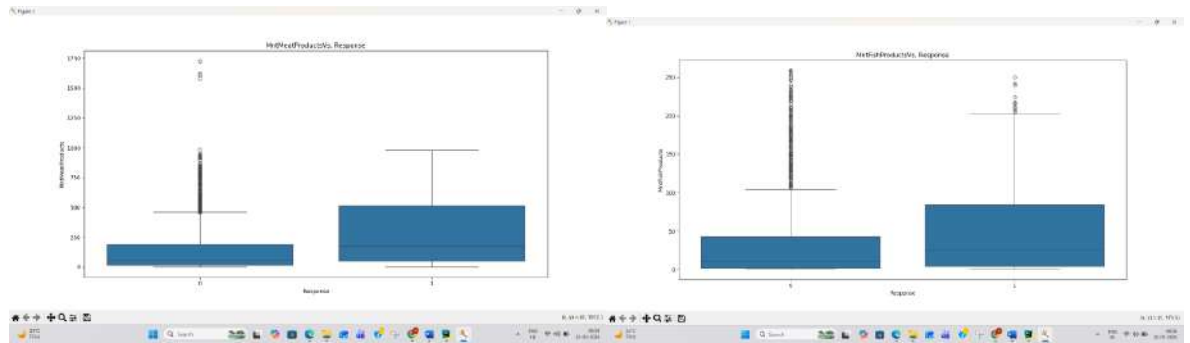
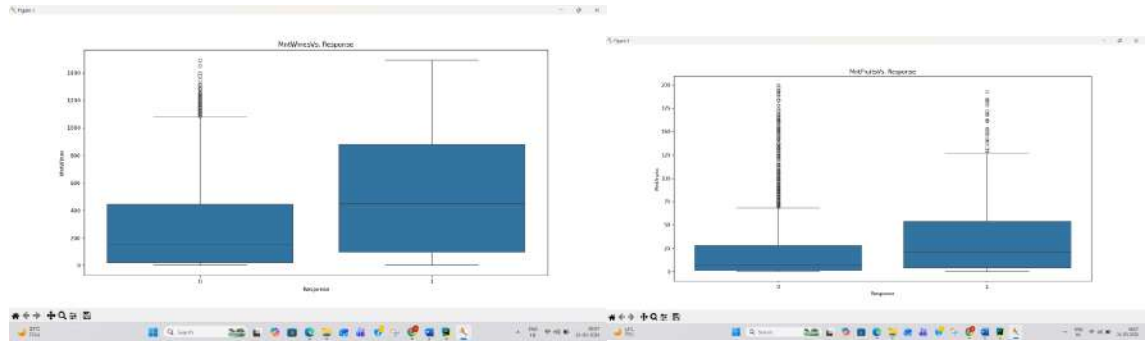
b. Distribution of Categorical Variables:

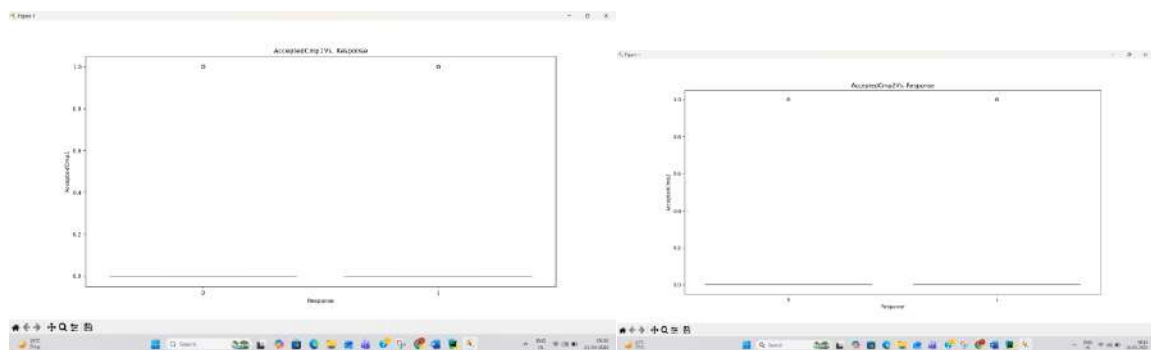
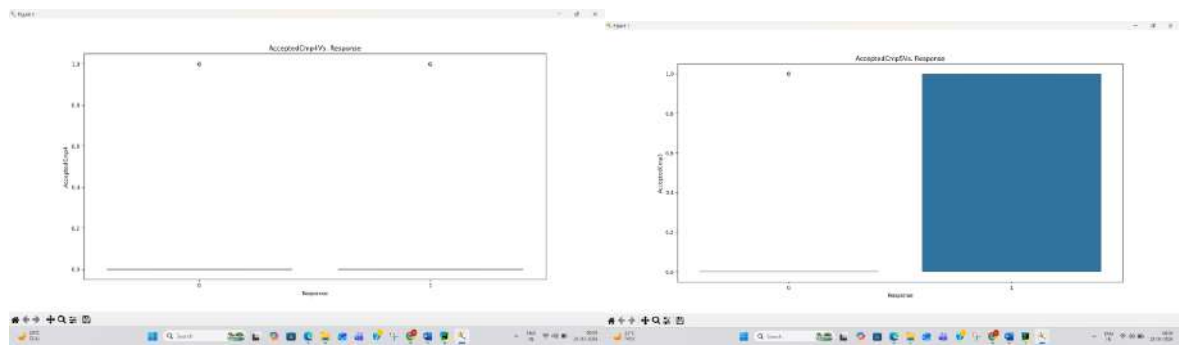
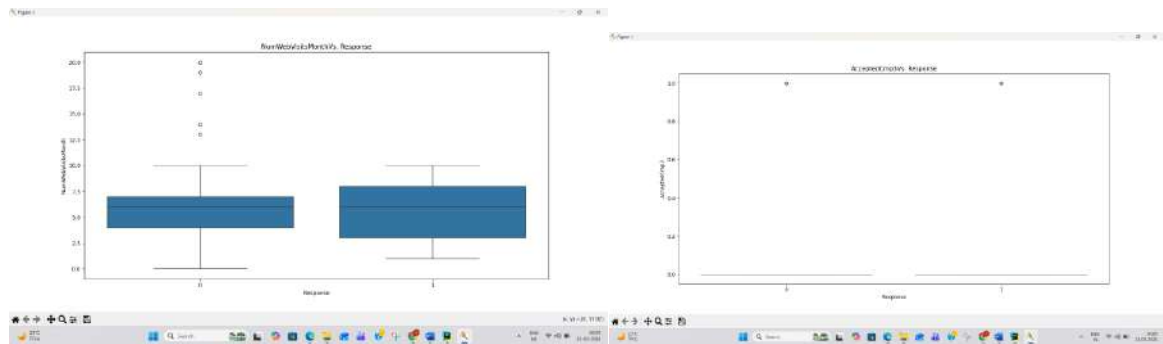
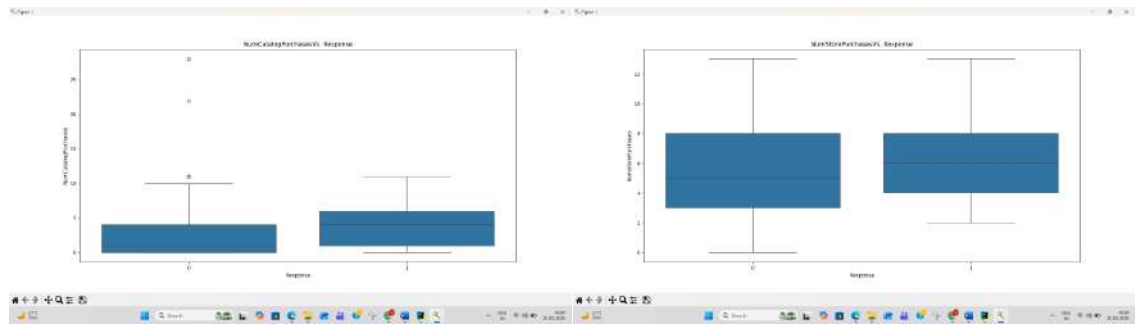
- Bar plots were employed to explore the distribution of each categorical variable, offering insights into the frequency of different categories.

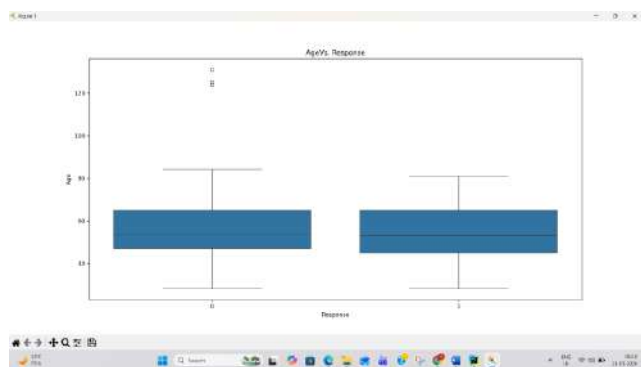
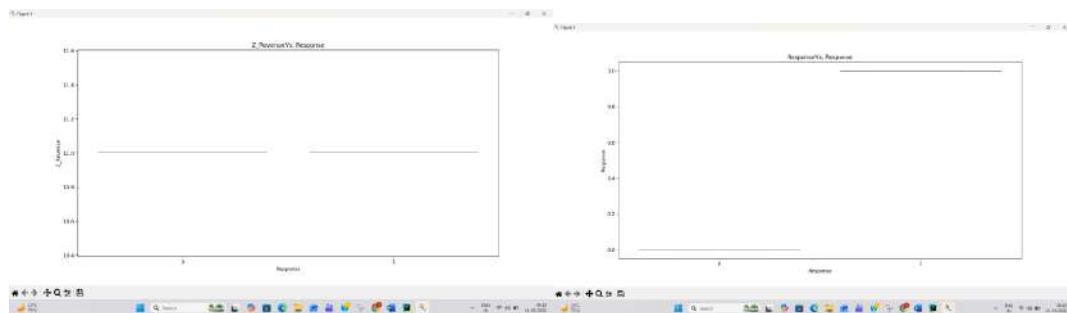
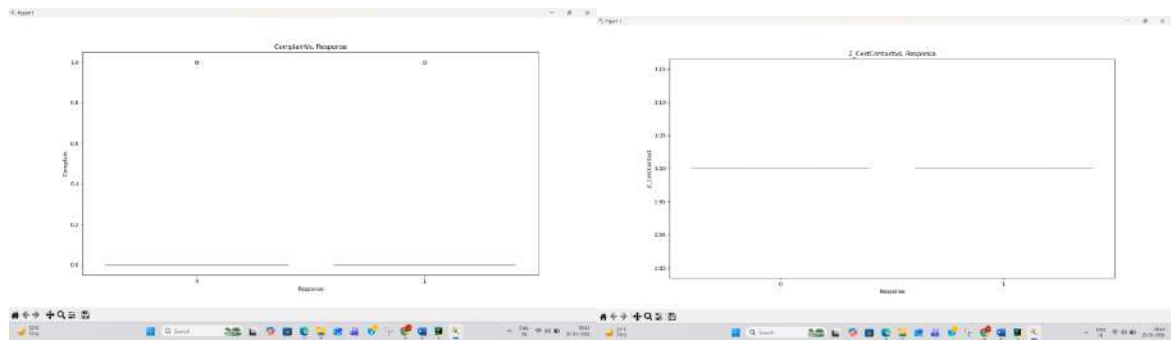
Task 4: Bivariate Analysis:

- Explore the relationship between numerical variables and the target variable (Response) using scatter plots or correlation matrices









B . Explore the relationship between categorical variables and the target variable using bar plots or chi-square tests

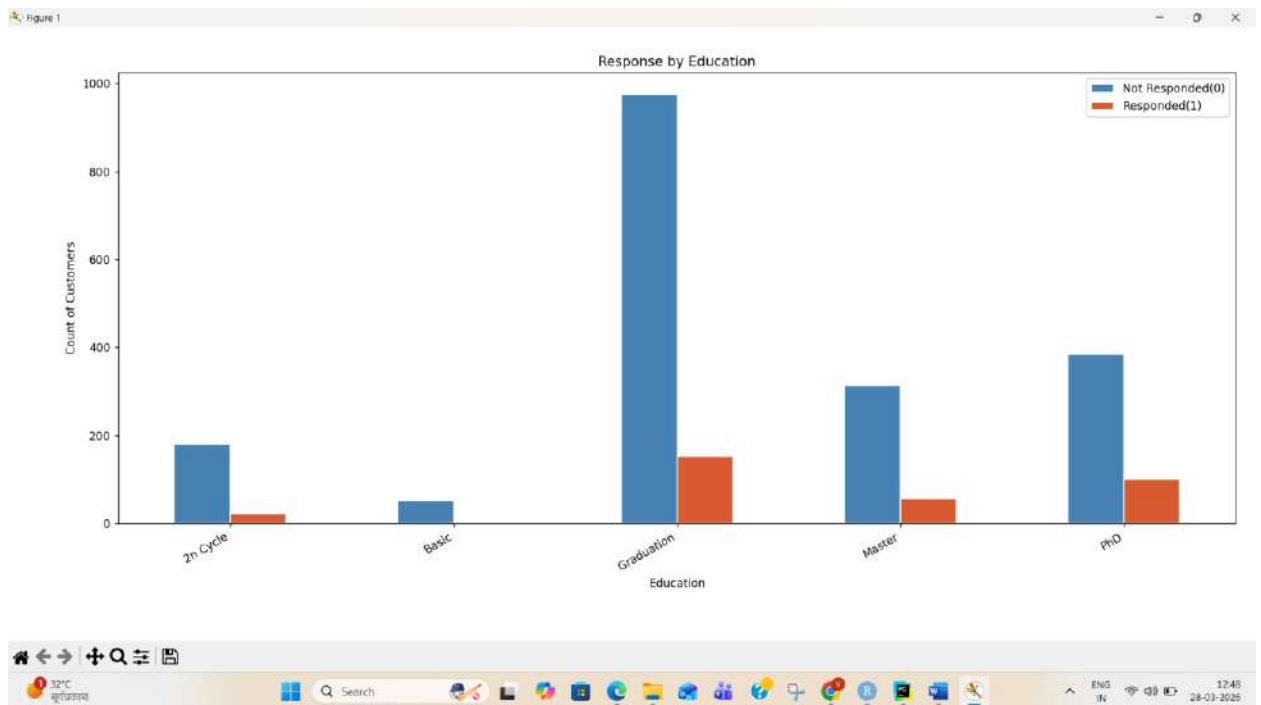
```
PythonProject - Version control - Current File - Unlock Pro

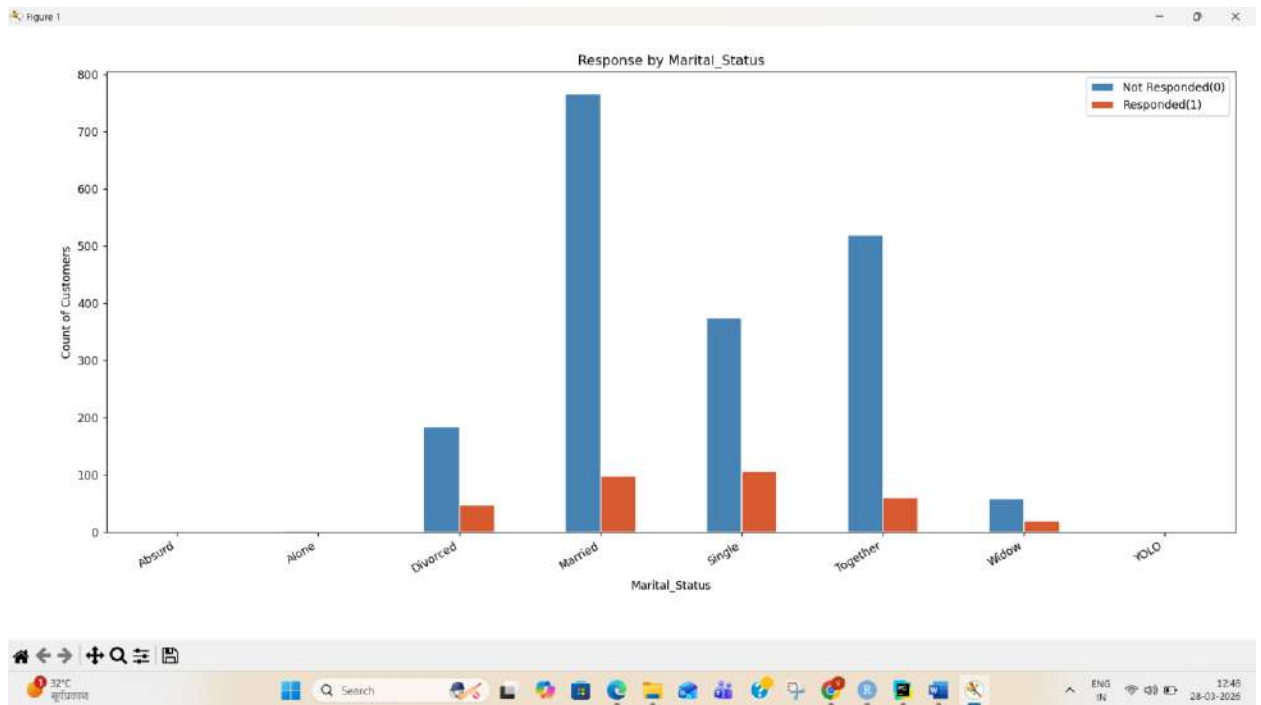
Project - PythonProject - Capstone_Project.py
  PythonProject
  .venv
  .venv\library root
  exportToHTML
  abcd.py
  AUS_Sales.csv
  Canada_Sales.csv
  Capstone_Project.py
  cleaned_data.csv
  Course.csv
  Data Dictionary - Response to marketing camp
  day 2.py
  Day 3.py
  Day 4.py
  Day 5.py
  Day 6.py
  Day 7.py

Capstone_Project.py
31 # plt.show()
32
33 cat_cols = ['Education', 'Marital_Status']
34
35 for col in cat_cols:
36     ct = pd.crosstab(df[col], df['Response'])
37
38     ct.plot(kind='bar', figsize=(9, 5),
39            color=['steelblue', '#D85A3D'],
40            edgecolor='white')
41     plt.title(f'Response by {col}')
42     plt.xlabel(col)
43     plt.ylabel('Count of Customers')
44     plt.xticks(rotation=30, label='right')
45     plt.legend(['Not Responded(0)', 'Responded(1)'])
46     plt.tight_layout()
47     plt.show()
48

Run - Capstone_Project.py
C:\Users\Hp\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Hp\PycharmProjects\PythonProject\Capstone_Project.py
Process finished with exit code 0

PythonProject - Capstone_Project.py 92-20 CRLF UTF-8 4 spaces Python 3.10 (PythonProject)
```





- 1 If one category (e.g. PhD) has a noticeably taller orange bar relative to its blue bar, that group responds better.
 - 2 If all bars look proportionally the same, the variable probably doesn't influence response.
- c. Explore the relationship between numerical and categorical variables using box plots or violin plots

```

70 # plt.figure(figsize = (10,5))
71 # sns.boxplot(x = Education, y = Income, data=df, palette='Set2', inner='quartile')
72 # plt.title('Income Distribution by Education')
73 # plt.xticks(rotation = 30)
74 # plt.tight_layout()
75 # plt.show()
76
77 plt.figure(figsize = (10,5))
78 sns.boxplot(x='Response', y='MntWines', data=df, palette=['steelblue', '#D85A30'])
79 plt.title('MntWines spending by campaign Response')
80 plt.show()

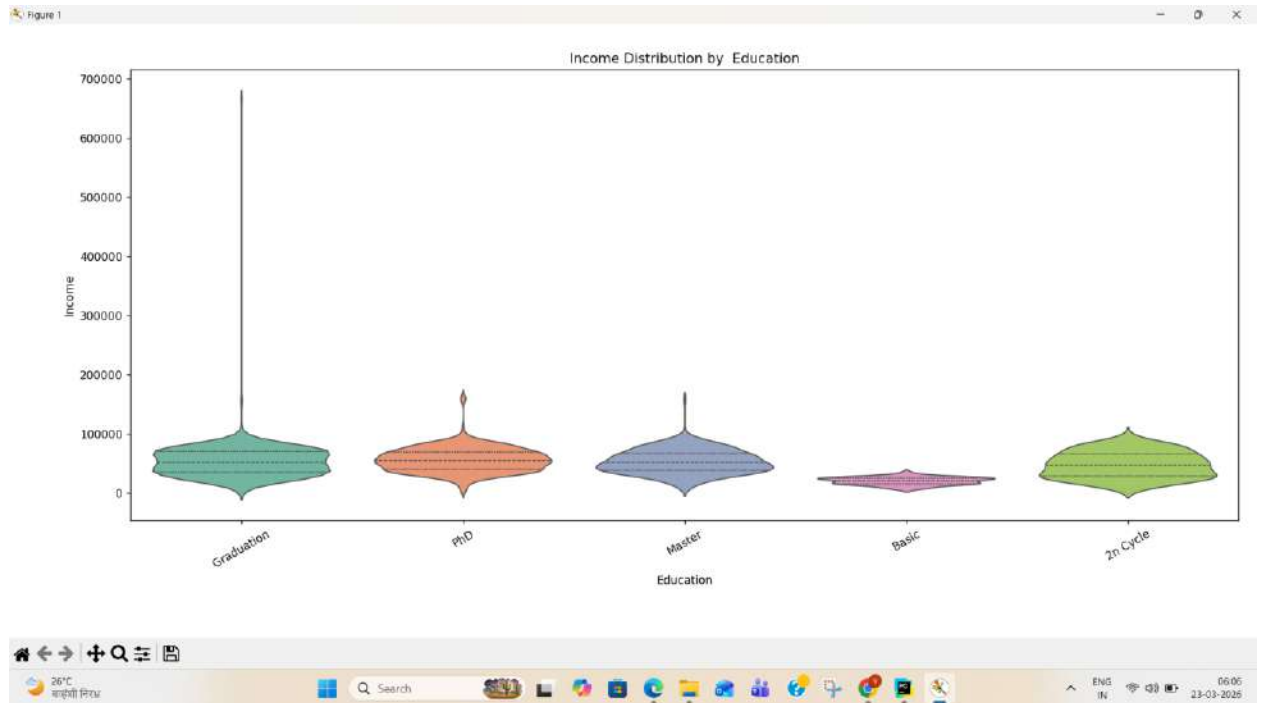
```

Warning: Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```

sns.boxplot(x='Response', y='MntWines', data=df, palette=['steelblue', '#D85A30'])

```

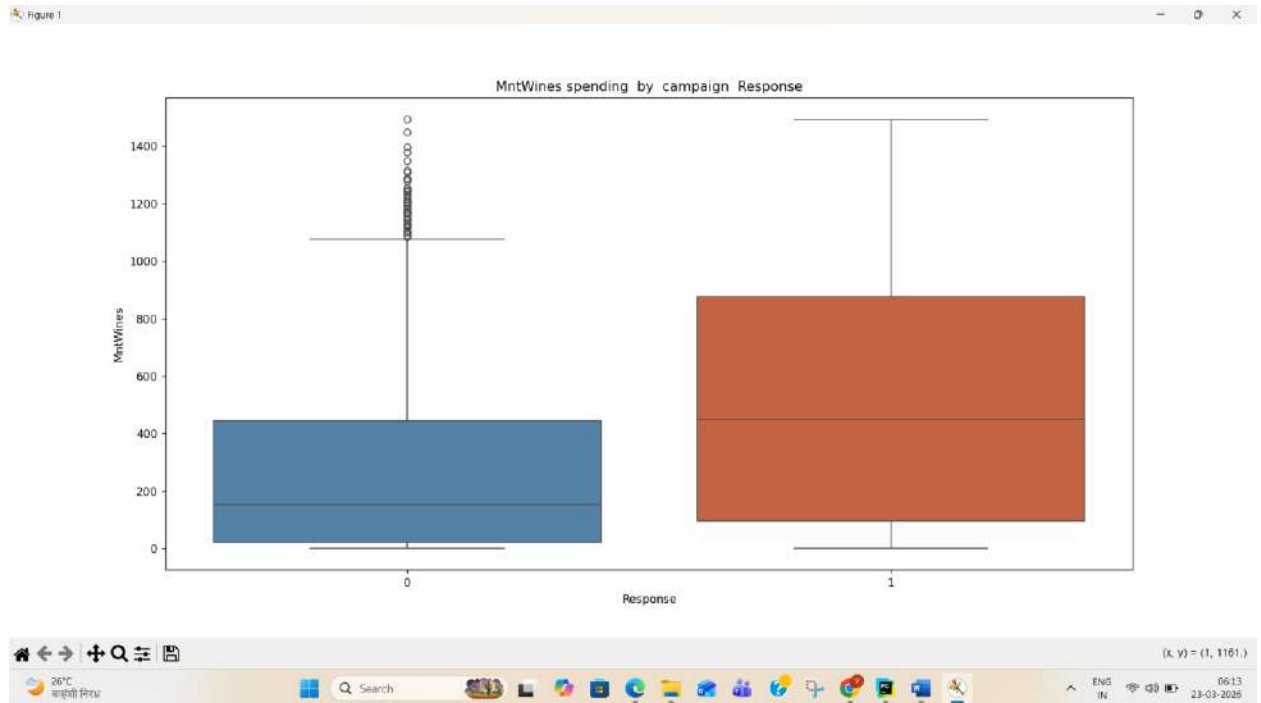


1 **Basic education = lowest income** — narrow violin confirms very little income variation

2 **Graduation has the widest income spread** — most diverse group with extreme outliers up to ~670K

3 **PhD & Master have similar income profiles** — both centered around ~50–70K with modest high-end spread

4 All groups share a **similar median income band (~40K–70K)**, but differ greatly in their upper tails



Customers who accepted the campaign spent significantly more on wine. This suggests wine spending is a **strong positive indicator** of campaign acceptance — a useful feature for predictive modeling.

Median wine spending is around **~450**, significantly higher

IQR spans roughly **~100 to ~850** — very wide spread

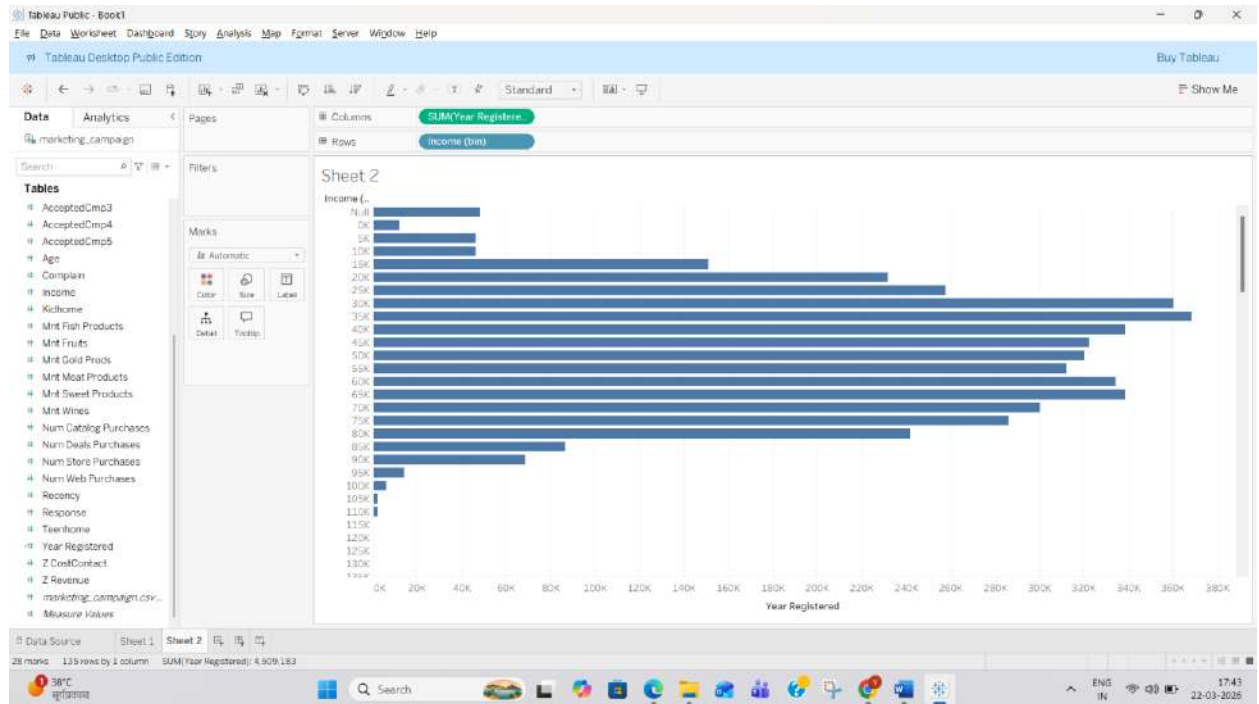
Tableau Tasks:

Task 1: Visualize Customer Income Distribution grouped by Year of Registration

Create a calculated field -Year Registered-year(Dt_Customer)

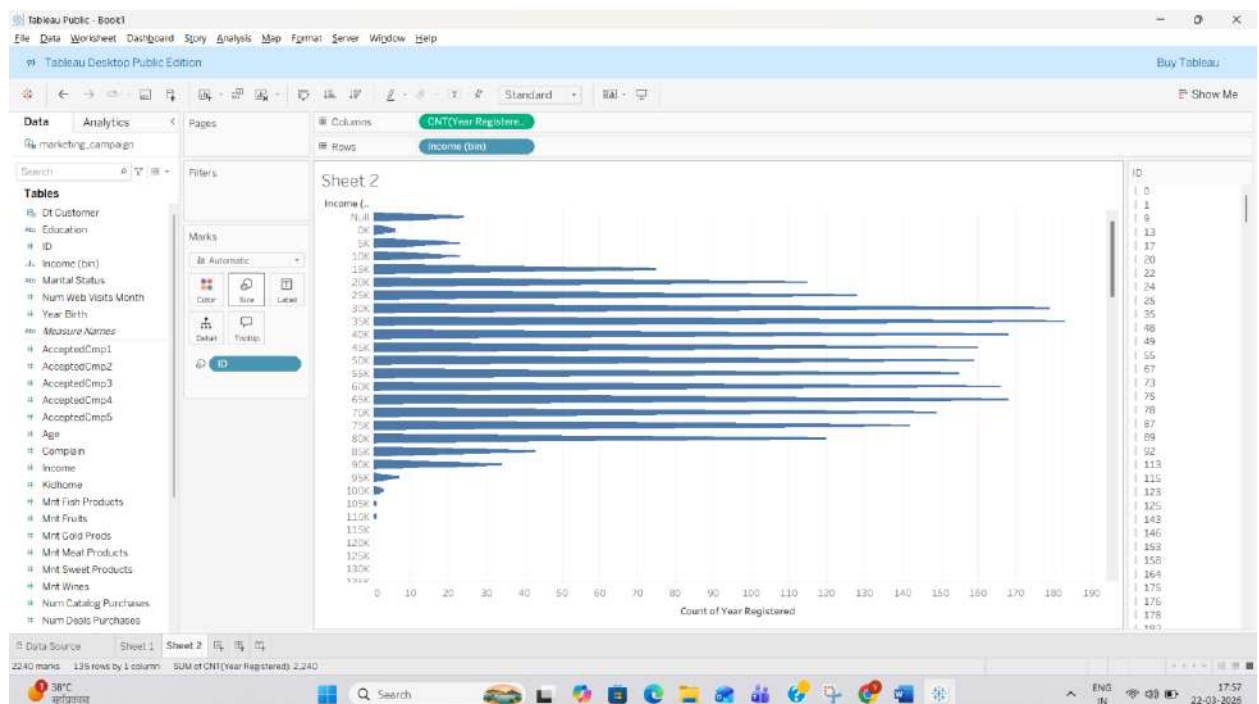
Create a bin of 5000 for income column

Drag Income bins to rows , drag Year registered calculated column to columns



From this , we came to know people earned 35k average and some people have zero income , and very few of them earned 1 lakh

Drag Id to size this will give kde effect



Rename to Yearly Income Distribution

Task 2: Examine the breakdown of education levels and marital status using side-by-side circles to represent count

Drag Education to color drag income to columns and education to label also

Drag Marital status to columns and also to color

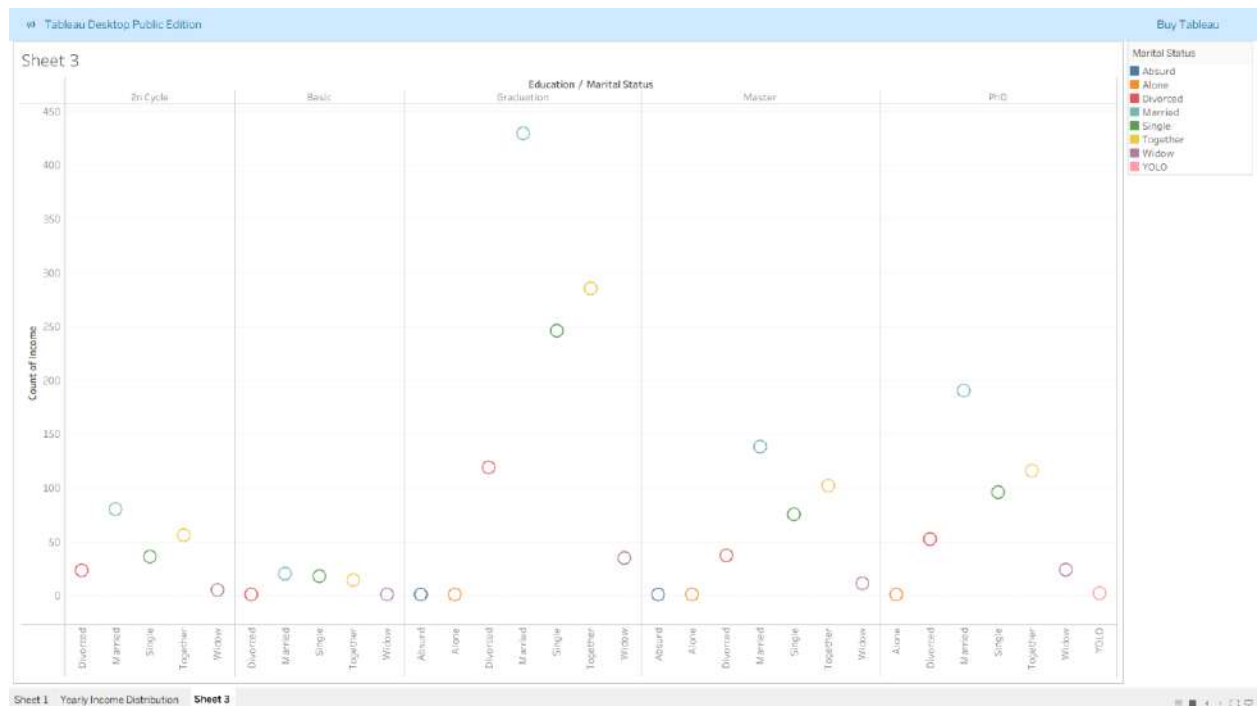
Change aggregation of sum to count

We have maximum people from married background

For side-by-side circles we will drag income to rows

Change aggregation to count

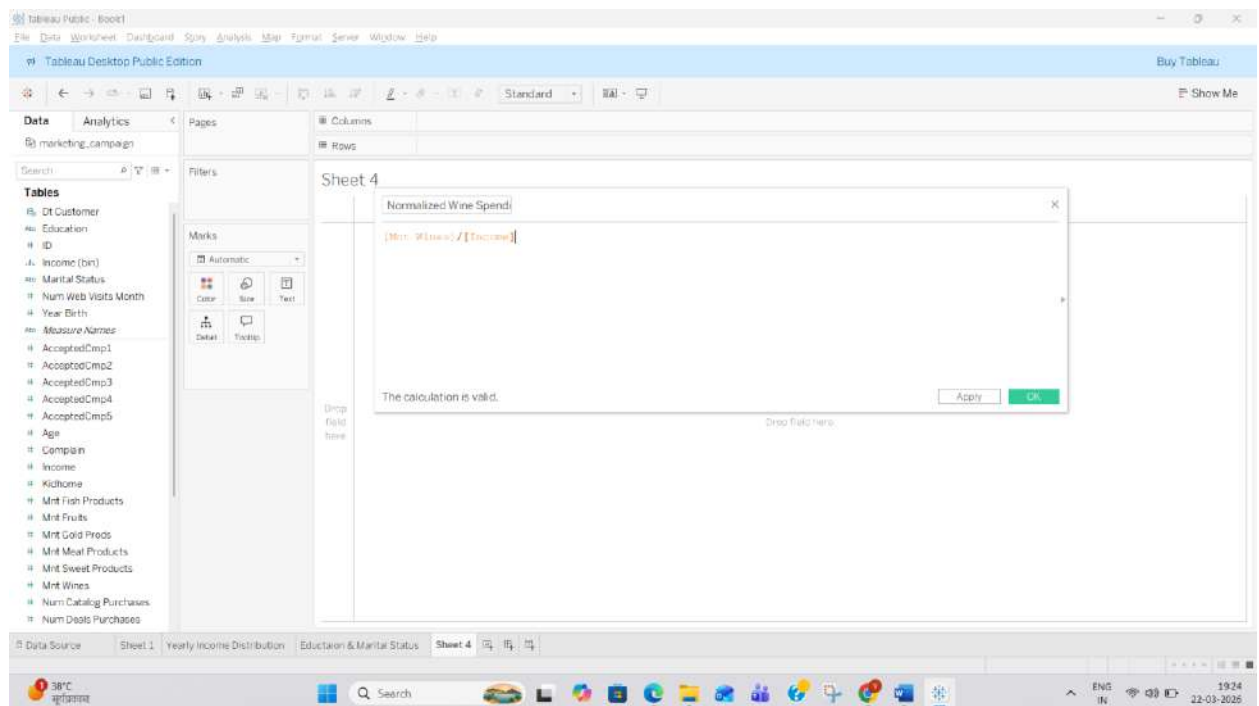
Select side by side circle



Married (green) has the highest count (~429) — tallest circle in the entire chart

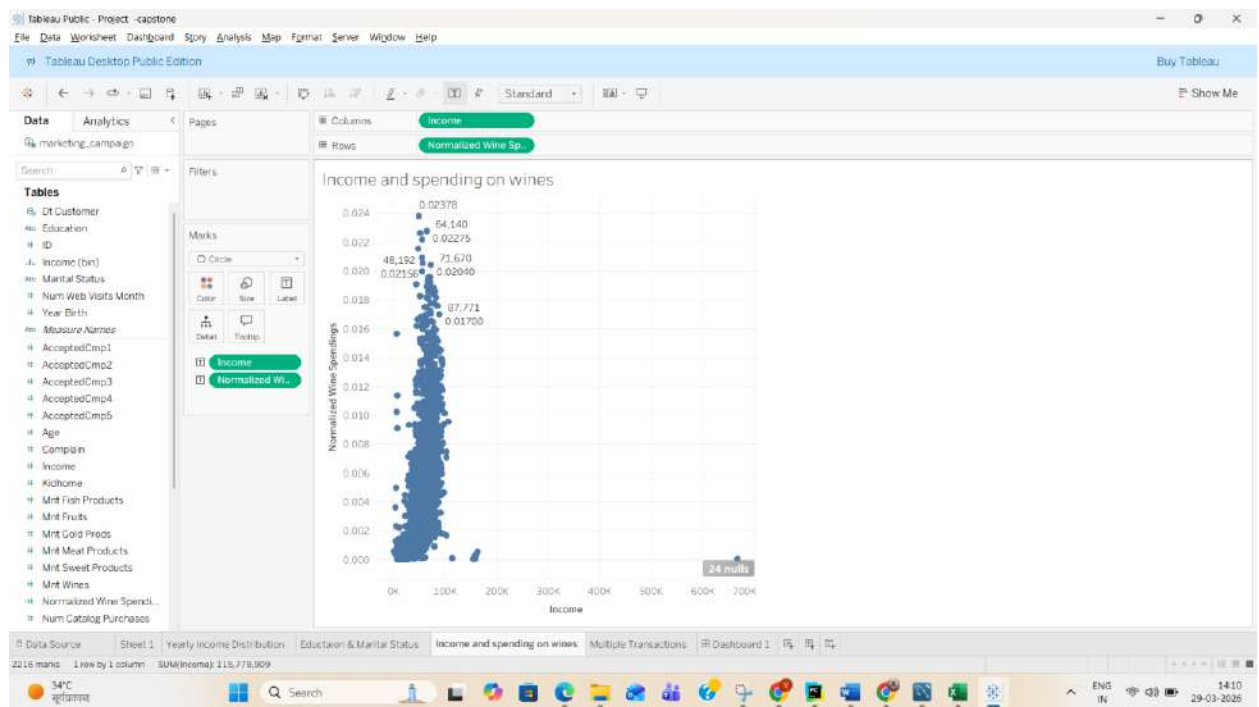
Graduation dominates all other education levels in volume

Task 3: Explore the relationship between Income and Wine Spending



Drag income to columns and normalized wines to rows and both are numeric automatically scattered plot created

Rename to Income and spending on wines



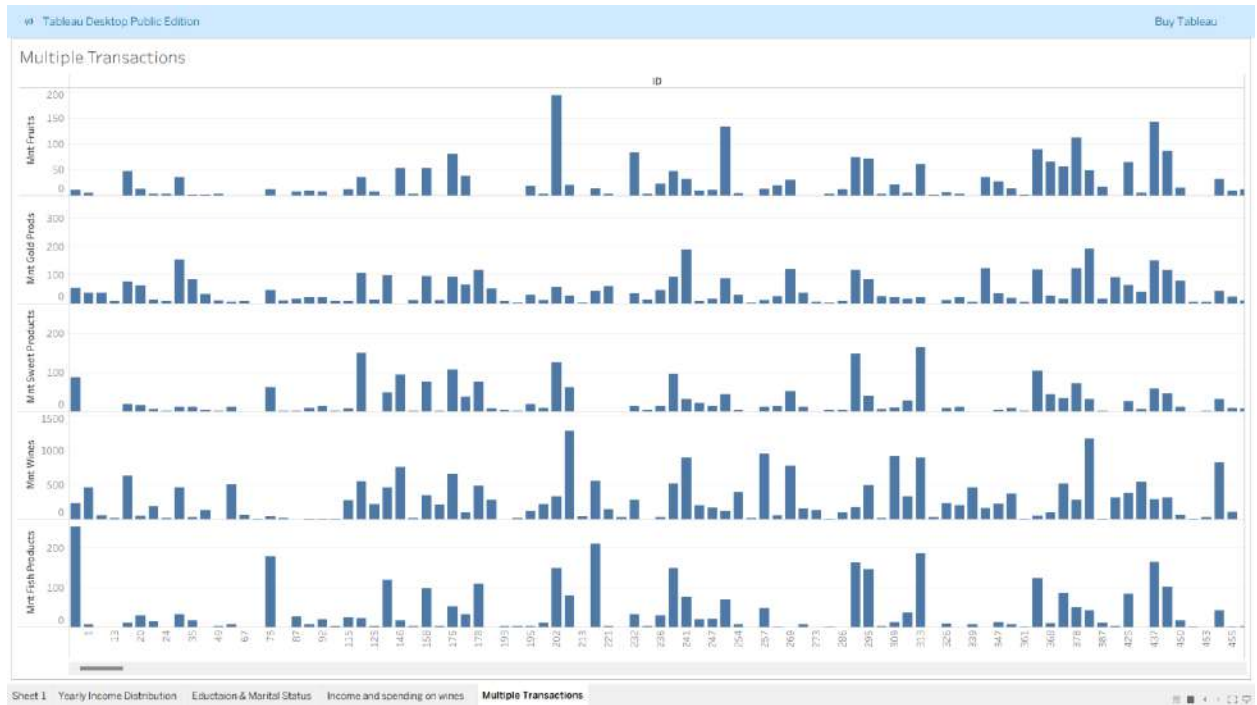
Wine spending is not strictly income-driven. The highest normalized wine spenders are actually in the **middle-income range** not among the wealthiest customers. This suggests **lifestyle/preference factors** may matter more than income for wine purchases.

Task 4: Analyze the frequency of purchases across various product categories

Drag Mnt wines, Mnt Meat Products, Mnt Fish Products Mnt Gold Prods , Mnt Fruits to rows

Drag Id to columns

Rename it to Multiple Transactions



MntWines is highest spending category all over charts

MntGoldProds is second highest spending and MntFruits has lowest

Most customers spend low amounts across all categories — bars are mostly near zero

Spending is inconsistent across customer IDs — no uniform pattern

Task 5: Create a dashboard using all the visualizations

Create automatic

Now we can put horizontal and vertical container

Drag all four charts to each container one by one

